Apollo-RTOS: An AGC-Inspired Real-Time Operating System for Embedded Systems

M. Ahsan Al Mahir

Department of Computer Science
University of Oxford

Supervisor: Prof. Alex Rogers

A project report submitted for the degree of Master of Mathematics and Computer Science

Trinity 2025

`9273` words
(using `texcount -inc -nobib -sum=1,1,0,1,0,0,0`)

Abstract

Despite half a century of advances, many challenges faced by early embedded computers like the Apollo Guidance Computer (AGC) remain highly relevant today. With only a 2MHz CPU and 4KB RAM, the AGC pioneered techniques in hybrid scheduling, fault-tolerant operation, and system recovery — capabilities dramatically demonstrated during Apollo 11's near-mission-abort alarms.

This project presents **Apollo-RTOS**, a real-time operating system for the BBC `micro:bit v1`, reimagining the AGC's resilient architecture in a modern context. Implemented entirely in C++20 and inspired by the AGC's radical "restart to recover" philosophy, Apollo-RTOS combines hybrid cooperative-preemptive scheduling, dynamic memory allocation, persistent file systems across Flash and FRAM, and a Unix-like shell interface. It supports up to 16 concurrent processes within a minimal monolithic kernel, with integrated error recovery and automated testing.

Apollo-RTOS demonstrates how historical system design principles, when revisited through the lens of modern embedded development, can create robust, extensible, and highly reliable systems under severe resource constraints.

Contents

§1 Introduction

1.1 Background & Motivation

Over the past century, computing capabilities have grown at an unprecedented rate. The **Apollo Guidance Computer** (AGC), a pioneering system developed for the Apollo program, operated with just a *2 MHz CPU*, *4 KB of RAM*, and *72 KB of ROM*. Despite its severe hardware constraints, the AGC's operating system introduced a series of innovations that remain remarkably relevant: a hybrid task scheduler combining cooperative and preemptive mechanisms, robust error detection with system-wide recovery, and efficient use of limited memory through careful task prioritization and synchronization.

One of the AGC's most groundbreaking design philosophies was its embrace of *restart-driven fault tolerance*. Rather than attempt complex error handling routines in hardware-constrained conditions, the AGC favored quick, controlled restarts to recover from faults, enabling resilience even under critical mission conditions—as dramatically demonstrated during the Apollo 11 lunar descent when 1201 and 1202 alarms threatened to abort the landing.

Today, a device like the `micro:bit`—costing less than £20—offers significantly greater raw computing power than the AGC. Yet in embedded systems, the fundamental challenges remain: *real-time responsiveness*, *reliability*, *fault tolerance*, and *resource efficiency* are still paramount, especially for mission- or safety-critical applications.

This project draws inspiration not only from the AGC's design but also from the modular robustness of modern Unix systems like Linux, and from the elegant abstraction facilities provided by the C++ Standard Library. The goal is to reimagine the AGC's philosophies for a modern embedded context: combining historical pragmatism with contemporary programming paradigms.

1.2 Project Summary

This work presents **Apollo-RTOS**: a fully-featured, real-time operating system for the `micro:bit` v1, deeply inspired by the AGC and informed by the design philosophies of Linux and C++20. Apollo-RTOS adapts the AGC's core concepts—most notably its hybrid cooperative-preemptive scheduling model and restart-driven recovery mechanism—while integrating modern operating system abstractions essential for contemporary embedded platforms.

Key architectural features include:

- A lightweight, deterministic **hybrid scheduler** capable of supporting up to 16 concurrent processes, with synchronization primitives ensuring race-free concurrent execution. [§ 4]
- A **fault-tolerant recovery system** based on restart groups, enabling processes to persist across resets or power failures. [§ 6]
- A modular **file system** that unifies flash and FRAM storage, providing durable, high-endurance storage critical for recovery and data integrity. [§ 5.2]

- A **Unix-like shell interface** for process management, file manipulation, and system interaction, offering a familiar development environment. [§ 9]
- An **automated testing framework** and build infrastructure to validate components, accelerate development, and enable systematic debugging. [§ 10]
- A **C++20-based implementation** leveraging RAII, templates, constexpr programming, and static typing, inspired by the robustness and clarity of the C++ Standard Library.

Apollo-RTOS is built around a *monolithic kernel architecture*, balancing simplicity and performance. Critical system services—including drivers for UART, I2C, and display peripherals—are designed as lightweight, reusable modules. File system operations are abstracted over a block device layer, supporting both high-endurance FRAM and flash memory with transparent access semantics.

Beyond reinterpreting the AGC's innovations, Apollo-RTOS demonstrates how combining minimalist principles with modern software engineering can produce resilient, extensible, and maintainable real-time operating systems. It offers a practical blueprint for building robust, modular systems under tight resource constraints—an increasingly valuable skill in the landscape of embedded and edge computing.

The complete source code for Apollo-RTOS is available at:

`https://github.com/AnglyPascal/apollo-rtos`

Listing 1.1: Landing screen of Apollo-RTOS on `minicom`

```
 _____           _ _          _____ _____ _____ _____
|  _  |___ ___| | |___    | __  |_   _|     |   __|
|     | . | . | | | . |   |    -| | | |  |  |__   |
|__|__|  _|___|_|_|___|   |__|__| |_| |_____|_____|
      |_|

 Welcome to Apollo RTOS!

boot level: boot
stats: n_flash = 23, n_boot = 5, n_power = 3, n_reset = 33

>> accel &
started [4] output to stdout

>>
```

§2 The Apollo Guidance Computer

The **Apollo Guidance Computer** (AGC) was an early digital computer developed in the 1960s to provide real-time navigation and control for the Apollo spacecraft. Despite operating under severe hardware constraints, it became a cornerstone of the Apollo program's success.

The AGC featured a *2.048 MHz* CPU capable of executing approximately *40,000 instructions per second*. It used a *16-bit word length*, with 15 bits for data and 1 parity bit. The memory system combined *2 kilowords* (4KB) of erasable magnetic-core RAM with *36 kilowords* (72KB) of fixed core rope ROM to store both software and mission data.

Astronauts interacted with the AGC through the **Display and Keyboard** (DSKY) unit—a 21-digit segmented display coupled with a 19-button keypad—enabling manual command input and real-time monitoring. This human-machine interface was critical for spacecraft operation, particularly during complex maneuvers.

Figure 2.1: Margaret Hamilton standing next to listings of the software she and her MIT team produced for the Apollo Project.

Given the severe resource limitations, the AGC operating system, known as **the Executive**, was engineered for maximum efficiency. It prioritized critical tasks, ensured graceful degradation under faults, and maintained system stability even after resets or failures.

The following subsections summarize the main architectural features of the AGC Executive, based primarily on O'Brien [3].

2.1 The Executive

The **Executive** was the core component responsible for allocating system resources such as CPU time and dynamic memory to processes. It implemented a **priority-based cooperative scheduling** algorithm, summarized as follows:

- The highest-priority process has exclusive access to the CPU.
- Lower-priority processes remain idle until the highest-priority process:
  - lowers its priority, or
  - enters `sleep` while waiting for an event.
- If a new process with higher priority is scheduled while another is running:
  - the new process waits until the current process lowers its priority or sleeps,
  - then immediately starts execution.

Core Sets Process metadata was maintained in a table called **Core Sets**, similar to modern process descriptors. Each active process occupied one Core Set. The system supported:

- 6 Core Sets in the Command Module, and
- 7 Core Sets in the Lunar Module.

Addressable Memory Each process was allocated 7 words of private memory, typically used for general computation. In interpreter mode, these acted as accumulator registers. Processes requiring more memory could request 42 additional words from the Vector Accumulator (VAC) area.

Starting a New Process When a process was created, it did not immediately preempt the CPU, even if it had higher priority. A special register, `NEWJOB`, indicated the ID of the highest-priority runnable process:

- `NEWJOB == 0` indicated the current process was the highest-priority.
- A nonzero `d` indicated that the process represented by Core Set `d` had higher priority.

Upon new process creation, `NEWJOB` was updated if the new process had higher priority than the currently running one.

Process Switching Context switches occurred only when the current process voluntarily yielded control. A running process was required to check `NEWJOB` at least every 20 ms to detect higher-priority contenders. Failure to check `NEWJOB` for more than 640 ms was treated as a stall, triggering a hardware restart.

During a switch, the current process's context was saved in Core Set 0, and the context of the new process, identified by `NEWJOB`, was restored. Execution then resumed automatically from the last instruction of the switched-in process.

If a process relinquished execution—either by sleeping or lowering its priority—the system

scanned the Core Set table to find and schedule the next highest-priority runnable process.

Sleep and Wakeup A process could enter `sleep` to wait for an external event. The `JOBSLEEP` instruction marked the process as sleeping by negating its priority, allowing the next eligible process to run.

When the awaited event occurred, another process could wake it using `JOBWAKE`, which reversed the priority negation. `JOBWAKE` also checked `NEWJOB` to determine whether the woken process is highest priority.

Processes could also delay themselves for a fixed duration using `DELAYJOB`, which scheduled an "alarm" via the **waitlist** mechanism.

The Waitlist The **waitlist** introduced preemptive capabilities to the otherwise cooperative Executive, allowing tasks to be scheduled with precise deadlines. Conceptually, it was a ***sorted linked list*** of tasks ordered by deadline. When a new task t was added, it was inserted such that:

- any task t_0 with an earlier deadline appeared before t, and
- any task t_1 with a later deadline appeared after t.

Every 10 ms, a timer interrupt checked if the earliest task had reached its deadline. If so, the task was executed and removed from the list. Waitlist tasks, executed inside the interrupt handler, were required to be brief—typically under 5 ms or 150–200 instructions.

2.2 Restart Mechanism

Despite its limited resources, the AGC was engineered to achieve high reliability, even in the presence of faults. Modern CPU and OS designs dedicate extensive resources to error detection and recovery; by contrast, the AGC lacked hardware support for sophisticated fault management. In particular, the absence of a memory protection unit made reliable detection of memory corruption nearly impossible.

Instead, the AGC adopted a radical but effective strategy:

Just restart everything.

Restarting the system at arbitrary points posed challenges, as critical tasks could be interrupted mid-execution. To mitigate this, critical processes defined explicit recovery behaviors in the event of a restart. Each program was assigned to one of six **restart groups**, with each group capable of recovering one process at a time. A group could configure its recovery procedure to one of four modes:

- **Inhibit group restart:** No processes in the group are restarted.
- **Restart last display:** Restore and display the last known screen contents.
- **Execute one restart routine:** Restart the process from the beginning with minimal cleanup.

- **Execute two restart routines:** Perform cleanup and resume execution from a designated recovery point.

To minimize overhead, recovery information was stored in a compressed format within the **phase tables**. During a restart:

- All active jobs and waitlist tasks were canceled.
- Interrupts were disabled, and system tables were cleared and reinitialized.
- The integrity of the restart phase table was verified, and recovery routines for each group were executed sequentially.

These mechanisms enabled the AGC to gracefully recover from a wide range of faults—an essential requirement for spacecraft systems where failure tolerance was critical to mission success.

2.2.1 The 1201 and 1202 Alarms During Apollo 11's Lunar Descent

The Apollo 11 lunar landing provided a dramatic demonstration of the AGC's resilience. During the final descent to the Moon, the Lunar Module's guidance computer triggered unexpected alarms—specifically, program alarms 1201 and 1202. These alarms indicated *executive overflows*, meaning the system's scheduler was overloaded and unable to complete all tasks within real-time constraints.

The root cause was traced to the rendezvous radar being left active during descent, introducing spurious data due to a hardware quirk [7]. This unexpected input consumed significant CPU time, reducing the availability of computational resources for other tasks. An additional command to display altitude data further strained the system, ultimately triggering the alarms.

Despite these conditions, the AGC's design allowed it to prioritize critical tasks. On detecting overload, the system gracefully shed nonessential processes through a controlled restart while maintaining key operations such as engine control and navigation. Mission Control evaluated the situation and, trusting the system's robustness, authorized the crew to proceed. Thanks to this resilience, Apollo 11 successfully landed on the Moon, transforming a potential abort into a historic achievement.

2.3 Interpreter

To extend its limited native instruction set, the AGC incorporated a custom virtual machine known as the **Interpreter**. The Interpreter operated as a *bytecode engine* layered atop the AGC's basic instruction set, enabling the execution of more complex *pseudo-instructions* while conserving memory and improving computational efficiency for critical tasks.

The Interpreter provided a broader set of operations, including arithmetic, trigonometric, vector and matrix calculations, data manipulation, and sophisticated branching. These ex-

tensions were essential for supporting the advanced navigation and guidance computations required during spaceflight.

While the AGC's native instruction set emphasized speed, the Interpreter introduced compact, high-level mathematical functionality without compromising system responsiveness.

2.3.1 Display and Keyboard

The AGC featured a distinctive human-machine interface called the *Display and Keyboard* (**DSKY**), which allowed astronauts to interact with the computer using a structured *verb-noun paradigm*. Commands were constructed by selecting a *verb* (action) followed by a *noun* (object), mirroring natural language. For example, the command "Verb 06 Noun 36" instructed the AGC to display the current time.

This consistent, intuitive interface enabled astronauts to monitor spacecraft systems, execute computations, and perform maneuvering operations precisely and reliably.

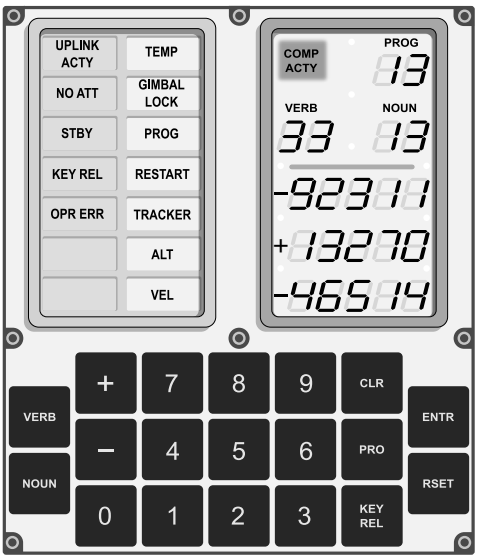

Figure 2.2: DSKY interface diagram by Oona Räisänen.

§3 Design of Apollo-RTOS

3.1 Inspiration

Apollo-RTOS draws inspiration from both the Apollo Guidance Computer (AGC) operating system and the modern Linux OS. The AGC OS was designed to meet specific mission requirements, allowing for a simplified and optimized design. In contrast, Linux is a general-purpose OS built to support diverse applications, resulting in a much more complex architecture.

The AGC operated in a tightly controlled hardware and software environment where all components could be trusted. Thus, isolating the hardware, OS, and application code was unnecessary. Additionally, the AGC lacked memory protection features, making kernel/user space isolation—a fundamental aspect of general-purpose OSs like Linux—impossible.

Apollo-RTOS adopts the same high-level design philosophy as the AGC: implement core concepts effectively in a resource-constrained environment, while applying contemporary engineering where beneficial. Key design decisions include:

Hardware Selection The `micro:bit` v1 was chosen as the primary target platform. Its Cortex-M0 CPU is among the smallest in the ARM family and closely resembles the AGC hardware.

Monolithic Kernel Design Following Linux's model, Apollo-RTOS adopts a monolithic kernel for simplicity and efficient execution in a constrained environment, at the cost of more challenging verification and testing.

No Kernel/User Space Isolation In microcontroller programming, the entire program—kernel and applications—is compiled into a single binary. Strict isolation would add unnecessary complexity.

Library Calls Instead of System Calls Since Apollo-RTOS does not enforce user/kernel space separation, direct library calls are used rather than system calls, simplifying the interface and reducing overhead.

Shared Memory IPC Although shared memory risks race conditions, it offers the fastest and simplest form of IPC and is thus adopted as the primary mechanism.

Memory Usage Philosophy While the `micro:bit` has limited memory, it offers far more than the AGC. Prioritizing maintainability over aggressive optimization, we follow Knuth's maxim that "premature optimization is the root of all evil."

We also drew inspiration from the `microbian` OS for the `micro:bit`, developed by Spivey [6], particularly for hardware headers, startup code, and interrupt handler structures.

3.2 Implementation Architecture

In this section, we briefly overview Apollo-RTOS's core modules and their interconnections:

Scheduler The centerpiece of the OS, managing:

- Process execution and context switching.
- Library functions for process creation, priority management, and synchronization.

Waitlist Implements a deadline-driven preemptive scheduler (similar to cron jobs), executing time-sensitive tasks at precise intervals.

Signals, Mutexes, and Barriers Enable low-overhead inter-process communication (IPC) and synchronization among processes.

Memory Allocator Handles all dynamic memory management. Allocators maintain freelists of memory chunks and serve both runtime heap and stack allocations.

File System Provides persistent storage through:

- A FRAM-based file system over I2C.
- An optional flash memory file system.
- Support for both memory-mapped and character-stream file access.

Error Handling Provides diagnostic logging and error detection, including runtime assertions and fault recovery.

Recovery Supports wrapping critical sections and processes with automatic recovery against different types of resets.

Drivers Interface with hardware components:

- **Timer**: Two 16-bit timers are configured for system clocking and profiling (`TIMER1` and `TIMER2`).
- **Serial**: Manages UART-based serial communication.
- **I2C**: Manages I2C master communication.
- **Display**: Controls the 5×5 LED matrix.

Shell Provides a Unix-like command-line interface for user interaction and process management.

Testing Framework Allows registering and running unit and subsystem tests in isolation during system startup.

"Standard" Library Offers utility data structures and functions, with interfaces inspired by the C++ Standard Library.

§4 Scheduling: Cooperative and Preemptive

At the core of Apollo-RTOS lies the scheduler. It is a simple, priority-based cooperative scheduler inspired by the AGC Executive. The highest-priority process retains exclusive control of the CPU until it voluntarily yields or lowers its priority. This design offers several important advantages:

- **Simplicity**: The scheduling algorithm is straightforward to implement and debug.
- **Determinism**: Execution order is predictable, making system behavior easier to reason about.
- **Fewer Race Conditions**: Since processes do not preempt each other, data races on shared variables are significantly reduced.

A global structure `cpu` tracks the currently running process (`curr_proc`) and the highest-priority process ready to run (`hi_proc`). When a new process is created, its priority is compared with `hi_proc` and updated if necessary.

A context switch occurs when the running process voluntarily yields, either by sleeping on a channel or setting a timeout. Alternatively, if the running process lowers its priority, the scheduler searches the process descriptor table for the next highest-priority runnable process.

> A process's priority is represented by two parameters: a *priority level* and a *weight*. Each time a process runs, its weight increments. To ensure fairness among processes with identical priority levels, the process with the lower weight is favored.

Listing 4.1: `include/core/procs.h`

```
struct priority_t {
  priority_lev_t lev = EMPTY;
  uint8_t weight = 0;

  friend auto operator<=>(const priority_t &lhs, const priority_t &rhs)
  {
    if (lhs.lev != rhs.lev)
      return lhs.lev <=> rhs.lev;
    return (rhs.weight) <=> lhs.weight;
  }
};
```

To guarantee that the system always has a runnable process, an ***idle process*** (Listing 4.13) with the lowest priority is registered during system startup. The idle process runs whenever no other process is runnable.

At runtime, processes are represented by `proc_t` structures, stored in a fixed-size process descriptor table. By default, this table contains 16 entries, allowing up to 15 concurrent user processes alongside the idle process. Each `proc_t` holds the execution context and metadata needed for scheduling.

Listing 4.2: `include/core/procs.h`

```
struct proc_t {
  priority_t  priority;  // Current priority
```

```
3   void       *param;     // Parameter passed to the process function
4
5   byte_t     *stk_ptr;   // Current stack pointer
6   byte_t     *stack;     // Start of the stack area
7   size_t      stk_sz;    // Stack size
8
9   // ... other fields
10 };
```

When a process finishes execution, its allocated resources are deallocated, and its descriptor is marked available for reuse.

4.1 Process Creation

Processes are created using the library function **reg_proc**. Its role is straightforward: find an available process descriptor, allocate a stack, and update the **hi_proc** variable if necessary.

Listing 4.3: src/core/sched.cpp

```
1 pid_t reg_proc(const proc_def_t *def, void *param)
2 {
3   auto &[name, priority, stk_sz, proc_func] = *def;
4
5   // allocate a process descriptor
6   proc_t *proc = procs.alloc();
7
8   // allocate and populate the stack frame according to § 4.2
9   stack_allocator.acquire(proc, stk_sz, proc_func, param, exit);
10
11  // raises priority, comparing against hi_proc
12  incr_priority(pid, priority);
13
14  // ...
15  return pid;
16 }
```

> A newly created process is treated as if it has just undergone a context switch. Thus, the stack must be pre-filled with the appropriate register values in the correct order, as detailed in § 4.2. Line 9 ensures this setup is performed correctly.

In line 12, **reg_proc** calls **incr_priority** to raise the new process's priority. This function also checks and updates **hi_proc** if the newly created process has a higher priority.

Listing 4.4: src/core/sched.cap

```
1 void incr_priority(pid_t pid, priority_t priority)
2 {
3   proc_t *proc = procs[pid];
4   proc->priority = priority; // set priority
5   if (cpu.hi_proc == nullptr || cpu.hi_proc->priority < priority)
6     cpu.hi_proc = proc;
7 }
```

Processes can call **incr_priority** to increase their own or another process's priority before entering a critical section. However, processes are expected to be mindful of system-wide

fairness: a high-priority process should lower its priority once it no longer needs exclusive control. This is done via **decr_priority**. When a process lowers its priority, the process descriptor table is scanned for the next highest-priority runnable process, and a context switch is triggered.

Listing 4.5: src/core/sched.cpp

```
void decr_priority(priority_t priority)
{
  cpu.curr_proc->priority = priority;
  cpu.hi_proc = procs.max_priority();
  change_proc();
}

void change_proc()
{
  if (cpu.hi_proc != cpu.curr_proc)
    /* trigger a PendSV interrupt, initiating context switch */
}
```

4.2 Context Switching

During a context switch, the execution context of the old process must be saved to its stack in a predetermined layout so that it can be restored later. Thus, we must first understand the stack layout on the Arm Cortex-M0.

When **change_proc** triggers the PendSV interrupt, the hardware automatically pushes key registers onto the stack in the following order (from lower to higher addresses):

Listing 4.6: Hardware saved registers

```
      new stack pointer -> |
                           |  r0
                    ^      |  r1
      stack grows   |      |  r2
      downwards     |      |  r3
      in memory     |      |  r12
                    |      |  lr   # Link register
                           |  pc   # Program counter
      old stack pointer -> |  psr  # Program status register
```

The remaining registers must be saved manually by the software. We chose the following layout for the software-saved context:

Listing 4.7: Software saved registers

```
      new stack pointer -> |
                           |  r8
                           |  r9
                           |  r10
                           |  r11
                           |  r4
                           |  r5
                           |  r6
   stack pointer after     |  r7
   hardware pushes regs -> |  lr_intr  # Return-from-interrupt address
```

The `PendSV` interrupt handler performs exactly this operation: saving the old process's context and restoring the new process's context.

Listing 4.8: `src/mpx.s`

```
@@@ PendSV handler for context switching
  .global pendsv_handler
  .thumb_func

pendsv_handler:

  @ Save current process state
  @@ by pushing software-saved registers onto the stack

  push {r4-r7, lr}      @ Save low registers and lr
  mov r4, r8            @ Copy high registers into r4-r7
  mov r5, r9
  mov r6, r10
  mov r7, r11
  push {r4-r7}
  mrs r0, msp           @ Move current stack pointer to r0

  @ Select new process
  bl cxt_switch

  @ Restore new process state
  @@ by popping software-saved registers from the stack pointed by r0

  msr msp, r0           @ Update stack pointer to new process stack
  pop {r4-r7}
  mov r11, r7
  mov r10, r6
  mov r9, r5
  mov r8, r4
  pop {r4-r7}
  pop {pc}              @ Pop lr into pc, jumping to the new process
```

The function **`cxt_switch`** swaps the old process's stack pointer with the new process's stack pointer:

Listing 4.9: `src/core/sched.cpp`

```
void *cxt_switch(void *stk_ptr)
{
  if (/* cpu.curr_proc is exiting */)
    /* release resources */
  else
    cpu.curr_proc->stk_ptr = (byte_t *)stk_ptr;  // Save old stack pointer

  cpu.curr_proc = cpu.hi_proc;
  return cpu.hi_proc->stk_ptr;  // Load new stack pointer
}
```

4.3 Process Suspension and Synchronization

> A ***runnable*** process is represented by a positive priority value. A process is marked as ***asleep*** by negating its priority.

This simple mechanism enables temporary suspension of a process without permanently changing its priority. A process can voluntarily give up execution in three main ways:

- **`yield`**`()`: The process checks whether it is the highest-priority runnable process. If not, it yields execution to the current highest-priority process.

Listing 4.10: `src/core/sched.cpp`

```
void yield()
{
  cpu.hi_proc = procs.max_priority();
  change_proc();
}
```

- **`sleep`**`(timeout)`: The process negates its priority to indicate an asleep state. It also registers an alarm (Listing 4.18) in the waitlist to wake it up after `timeout` milliseconds. A `timeout = 0` indicates indefinite sleep.

Listing 4.11: `src/core/sched.cpp`

```
bool sleep(time_t timeout = 0)
{
  if (period == 0)
    /* suspend indefinitely until someone else wakes it up */

  /* register an alarm (Listing 4.18) to wake it up after timeout ms */
  // negate priority to indicate asleep state
  decr_priority(-cpu.curr_proc->priority);

  /* return true if woken up by the alarm, false otherwise */
}

void wakeup(pid_t pid)
{
  incr_priority(pid, -procs[pid]->priority);
}
```

- **`wait`**`(chan, timeout)`: The process goes to sleep, waiting for a notification on the channel `chan`. If no notification arrives within `timeout` milliseconds, the process wakes up automatically and handles the timeout as an error.

  A channel is implemented as a fixed-length FIFO queue: processes that enqueue first are serviced first.

Listing 4.12: `include/core/sync.h`

```
bool wait(auto &chan, time_t timeout = 0)
{
  /* return error if the channel is full of waiting processes */

  // enqueue in the channel to be woken up by notify()
  chan.enqueue(curr_proc::pid());
```

```
  return !sleep(timeout);
    // the alarm should not wake up the process, indicates an error
}

void notify(auto &chan)
{
  /* return if nobody is waiting on channel */

  // wake up the first process waiting on channel
  wakeup(chan.dequeue());
}
```

4.3.1 Idle Process

At times, all processes may be suspended, awaiting external events. To prevent the CPU from being left idle without supervision, a dedicated ***idle process*** is registered at system startup. The idle process has the lowest possible priority and runs whenever no other process is runnable. It continuously checks if any higher-priority process becomes ready to execute.

Listing 4.13: `src/core/sched.cpp`

```
// defines the idle process with the lowest priority IDLE
PROC(idle, IDLE, 0, p)
{
  while (true)
    change_proc();
}
```

4.3.2 Synchronization

The **`wait`**/**`notify`** primitives form the backbone of the **`mutex`** implementation. Mutexes provide mutual exclusion for protecting shared resources. Critical components such as shared memory, file descriptors, and recovery phase table entries are guarded using mutexes.

Listing 4.14: `include/core/sync.h`

```
class mutex
{
  mutable chan_t lock_chan;
  mutable bool busy = false;

public:
  void lock() const
  {
    while (busy) // if the mutex is locked, queue in the lock_chan
      sched::wait(lock_chan);
    busy = true;
  }

  void unlock() const
  {
    busy = false; // unlock the mutex by waking up a waiting process
    sched::notify(lock_chan);
  }
```

```
19 };
```

To simplify usage, a wrapper RAII class `lock_guard` is provided. `lock_guard` acquires the mutex in its constructor and releases it in its destructor, ensuring that the mutes is always properly released when exiting a critical section.

4.3.3 Waiting for Child Processes to Terminate

Apollo-RTOS does not impose a strict parent-child process hierarchy as in Linux. However, it is often necessary for a process to wait for another process—potentially one it spawned—to terminate before continuing execution.

This is achieved through a synchronization primitive called `barrier_t`, implemented internally using `mutex`. A barrier blocks the current process until all processes it is waiting on have terminated.

Suppose process `p0` needs to wait for processes `p1`, `p2`, and `p3`:

- `p0` creates a `barrier_t` of size 3 and registers this barrier to the process descriptors of `p1`, `p2`, and `p3`.
- It calls `acquire` on the barrier for each process, going to sleep after the final `acquire`.
- When any of `p1`, `p2`, or `p3` terminates, the cleanup routine checks for an associated barrier and calls `release` on it.
- Once all registered processes have terminated, the barrier is fully released, and `p0` is woken up to continue execution.

This mechanism is encapsulated in an overloaded function `wait`, which allows a process to wait on multiple `pid_t`'s simultaneously:

Listing 4.15: `include/core/sync.h`

```
1  // waits for multiple processes to terminate within a timeout,
2  // returns false if any awaited process does not terminate in time.
3  template <typename... Args>
4    requires(std::same_as<remove_ref_cv_t<Args>, pid_t> && ...)
5  bool wait(time_t timeout, const Args &...args)
6  {
7    // create a barrier sized to the number of awaited processes
8    barrier_t bar{sizeof...(args), timeout};
9
10   // lambda to register the barrier with a single process
11   auto lam = [&](auto arg) {
12     /* attempt to acquire a barrier slot for the process with pid 'arg' */
13     return bar.acquire(arg);
14   };
15
16   // attempt to acquire the barrier for each process,
17   // returns true if all processes terminated within the timeout,
18   // returns false if any process failed to terminate in time.
19   return (lam(args) && ...);
20 }
```

Applications of Barrier-based Waiting The process waiting mechanism is utilized in several key areas:

- **`shell`**: To wait for a spawned foreground process to complete execution.
- **`pkill`**: To wait until the target process is fully terminated.
- Unit tests: To wait for multiple spawned processes to terminate before verifying test outcomes.

4.4 Waitlist: Preemptive Scheduling

The waitlist is a timer-driven list of tasks scheduled to execute after a specified interval. The algorithm we use is nearly identical to that of the AGC, except we implement it using a sorted linked list rather than a static array.

Listing 4.16: `src/core/waitlist.cpp`

```
struct waitlist_t {
  time_t remaining = 0;
  runnable_t func = nullptr;
  void *param = nullptr;
  waitlist_t *next = nullptr;  // Linked list link
};
```

If a task t_0 is scheduled to run in n_0 ticks, it is placed between tasks t_1 and t_2 with deadlines n_1 and n_2 such that $n_1 \leq n_0 \leq n_2$. We then set

$$t_0.\texttt{remaining} = n_0 - n_1$$

Thus, the time until t_0 executes is the sum of the `remaining` values of all preceding tasks.

Tasks are registered to the waitlist via:

```
void reg(string name, time_t interval, runnable_t func, void *param);
```

When the deadline for the first task in the list expires, it is simply popped off the list. The `remaining` field of the new head then indicates how long until its deadline expires.

Waitlist tasks are checked and executed at a fixed interval of `update_interval = 8ms`. When this interval elapses, the timer interrupt checks if there are any outstanding tasks. If so, it sets up a temporary stack to isolate the waitlist tasks and calls `waitlist::run()`.

The function `waitlist::run()` simply decrements the `remaining` field of the first task by `update_interval` and checks if the task has expired. If so, it executes all tasks with expired deadlines sequentially.

Listing 4.17: `src/core/waitlist.cpp`

```
void run()
{
  auto head = &waitlist_hd;
  if (head->next != nullptr) // there is a pending task
    head->next->remaining -= update_interval;

```

```
  // run tasks whose deadline has expired
  while (head->next != nullptr && head->next->remaining == 0) {
    auto task = head->next;
    head->next = task->next;

    auto [name, remaining, func, param, next] = *task;
    func(param);
    dealloc(task);
  }
}
```

The alarm tasks used by the scheduler to wake up sleeping processes after a timeout (Listing 4.11) are registered to the waitlist using the same mechanism.

Listing 4.18: `src/core/sched.cpp`

```
void default_alarm(void *ptr)
{
  auto proc = (proc_t *)ptr;
  if (/* the process is already awake */)
    return;
  incr_priority(procs.pid(proc), -proc->priority.lev);
}

// it is then used in sched::sleep
bool sleep(time_t timeout = 0)
{
  ...
  waitlist::reg(proc->name(), timeout, default_alarm, (void *)proc);
  ...
}
```

4.5 Signal Handling

Apollo-RTOS provides a lightweight, cooperative signal handling mechanism to support asynchronous inter-process communication and process control.

Signals are defined as an enumerated type `signal_t`, with predefined values:

- **`SIGTERM`**: Request process termination.
- **`SIGKILL`**: Force immediate process termination.
- **`SIGMSSG`**: User-defined message signal.
- **`SIGDUMP`**: Request process state dump.

Each process maintains an instance of `signals_t` within its descriptor, which tracks pending signals and their associated handlers:

```
struct signals_t {
private:
  uint32_t raised = 0;
  signal_handler_t handlers[N_SIGNALS] = { /* default handlers */ };

public:
  // Raise a signal
```

```
  void send(signal_t sig) { raised |= (1 << sig); }

  // Handle all raised signals
  void handle_signals();

  // Swap a signal handler and return the old one
  signal_handler_t swap(signal_t sig, signal_handler_t new_handler);

  // Clear all pending signals
  void reset() { raised = 0; }
};
```

Each signal is initially associated with a default handler, implemented via template specializations of `default_handler<sig>()`. Processes can override handlers dynamically using `swap_handler(signal_t sig, signal_handler_t new_handler)`.

Sending and Handling Signals Processes can asynchronously raise signals in other processes by calling `send_signal(pid_t pid, signal_t sig)`. Signals are handled cooperatively: when a process is context-switched in, its pending signals are processed before normal execution resumes.

This mechanism is integrated at the assembly level inside the PendSV interrupt handler (`pendsv_handler`). Before restoring the new process context, the system branches to **`handle_signals`**. Specifically, the call is inserted after restoring registers in Listing 4.8:

Listing 4.19: `src/mpx.s`

```
@@@ PendSV handler for context switching
pendsv_handler:
  @ Save current process state
  ...

  @ Select new process
  bl cxt_switch

  @ Restore new process state
  ...
  pop {r4-r7}

  bl handle_signals    @ handle signals before jumping into the new process

  pop {pc}             @ Pop lr into pc, jumping to the new process
```

The **`handle_signals`** function processes all raised signals for the incoming process (accessible via **`cpu.curr_proc`**), invoking the corresponding handlers:

Listing 4.20: `src/core/sched.cpp`

```
__extern_C__ void handle_signals(void)
{
  cpu.curr_proc->signals.handle_signals();
}
```

Process Termination via Signals Process termination is implemented using signals. The shell command **`pkill`** sends a **`SIGTERM`** signal to the target process. The default handler for **`SIGTERM`** sets the `term_req` flag within the process descriptor:

Listing 4.21: `src/core/sched.cpp`

```
template <>
void default_handler<SIGTERM>(void)
{
  cpu.curr_proc->term_req = true;
}
```

Well-behaved processes are expected to check this flag periodically within their main execution loops:

```
...
while (!curr_proc::term_req()) {
    // process work
}
...
```

Upon detecting a termination request, the process can perform cleanup and exit gracefully.

§5 Memory Management and File System

5.1 Dynamic Memory Allocation

A central component of Apollo-RTOS is its memory management system, which employs pool allocators for all runtime allocations. This design provides simple, efficient, and predictable dynamic memory management.

- The allocator maintains a linked list of free memory chunks.
- It searches the free list for a sufficiently large chunk during allocation.
- If no suitable chunk is found, it requests additional memory from the global allocator.
- Freed chunks are returned to the free list for reuse.

The allocator implementation (Listing B.2) is based on a doubly linked freelist (Listing B.1) with statically allocated nodes, avoiding dynamic memory allocations for list operations. It exposes a simple interface:

```
1 template <allocator_t alloc_func, size_t alignment>
2 class allocator {
3 public:
4     byte_t *alloc(size_t sz);
5     void dealloc(byte_t *ptr);
6 };
```

Apollo-RTOS provides two separate sets of memory allocation interfaces:

- **`kmalloc`** and **`kfree`** for kernel-space allocations.
- **`malloc`** and **`free`** for process-space allocations.

Each process descriptor maintains a linked list of all memory blocks allocated through **`malloc`**. This design serves two critical purposes:

- It ensures proper memory cleanup at process termination—automatically reclaiming any blocks not explicitly freed.
- It provides basic memory accounting and isolation between processes, despite the absence of hardware memory protection on the Cortex-M0.

The combination of kernel and process-specific allocators, along with explicit tracking of allocations per process, creates a robust memory management system that prevents memory leaks and maintains strict separation between memory domains. More details on the actual implementation can be found at Listing C.1.

5.2 File System

Apollo-RTOS implements a dual-storage architecture that leverages both the `micro:bit`'s onboard flash and an external FRAM chip, each serving distinct roles based on their respec-

tive characteristics. This design carefully balances performance, endurance, and access patterns to provide an efficient and reliable storage solution.

Flash Memory The 256KB onboard flash serves as the primary storage medium, with approximately 90KB occupied by the Apollo-RTOS binary. Flash memory offers direct CPU access for fast read operations but imposes significant write constraints. Writes require erasing entire 1KB pages (22.3 ms) before programming (20 μs per byte) [4], making small, frequent updates inefficient. Its endurance of approximately 20,000 erase cycles and 10-year data retention makes it ideal for storing relatively static data that is read often but modified infrequently, such as operating system binaries and configuration files.

FRAM Storage The MB85RC2567 FRAM chip [2] complements flash memory by offering superior endurance and write performance. Supporting up to 10^{12} write cycles and 95+ years of data retention, the 32KB FRAM is ideal for frequently updated data. Although accessed over I2C at 100KHz—resulting in lower read throughput compared to flash—it provides near-instantaneous write operations without the need for erase cycles. Being an external device, FRAM introduces I2C bus overhead and contention, which must be managed carefully at the system level.

Unified File System Design Rather than maintaining separate file systems, Apollo-RTOS introduces an abstract block device layer that unifies access across storage media. Each storage device is described by a `fs_desc_t` structure (Listing C.2) capturing device-specific parameters (e.g., block size, capacity) and read/write operations. A template-based approach enables compile-time optimization of storage access while maintaining a clean, extensible interface.

The file system is implemented entirely in headers under `include/fs/`, and uses a two-level table structure with comprehensive metadata tracking. This design enables efficient file management across heterogeneous storage devices, abstracting away differences in performance and endurance at the application level.

5.2.1 File System Backend

The file system backend manages physical storage using two core data structures: an inode table and a compact block allocation bitmap. The `inode_t` structure represents each file's metadata and storage allocation, while the `bitset` (Listing B.3) efficiently tracks free blocks across the storage device.

Each file is physically represented as follows:

Listing 5.1: `include/fs/fs_bck.h`

```
template <typename desc_t, desc_t desc>
struct inode_t {
  using addr_t = typename desc_t::addr_t;
  using blk_addr_t = uint8_t;
    // in either flash or FRAM, blocks are addressed with 8-bit integers

  blk_addr_t blks[desc.max_num_blks]; // non-contiguous block ownership
  mutable addr_t end;   // logical end of file
```

```
  const uint8_t nblks;  // number of blocks assigned
  mutable uint8_t flag; // file status encoded as a byte
};
```

Files are referenced system-wide using their inode number, defined as `fn_t = uint8_t`. The backend exposes the following public interface:

Listing 5.2: `include/fs/fs_bck.h`

```
template <...>
class fs_t
{
public:
  void format();
  void mount();    // loads header; formats if invalid
  void umount() const; // stores header back to device

  // Open a file using Linux-like flags (O_CREATE, O_WRITE, O_SHARED, etc.)
  pair<const inode_t *, bool> open(fn_t fn, size_t sz, uint32_t flags);

  void remove(fn_t fn, bool forced = false);

  // Load contents into a buffer
  template <bool sync = ASYNC>
  size_t load(const inode_t *inode, void *buf, size_t buf_sz, size_t off);

  // Store buffer contents into a file
  template <bool sync = ASYNC>
  size_t store(const inode_t *inode, const void *buf, size_t buf_sz, size_t off);
};
```

The detailed implementation of **`open`**, **`load`**, and **`store`** is presented in Listing C.3.

5.2.2 File System Frontend

The file system frontend acts as the primary interface between user processes and physical storage. It wraps raw `inode_t` structures into manageable file objects and implements features such as access control, memory mapping, and streaming operations.

At its core lies the **file descriptor table**, maintaining state for all open files. Each descriptor (`fd_t`) tracks:

- A reference to the underlying `inode_t`.
- Read/write access counters for concurrency management.
- Locking mechanisms for thread safety.

Descriptors are wrapped by `file_t` handlers, which:

- Enforce access permissions.
- Synchronize concurrent accesses.
- Automatically release resources upon closure.

Memory Mapping Subsystem Apollo-RTOS supports runtime memory mapping of file contents, enabling two modes:

- **Private mapping**: Local to the process; modifications are isolated.
- **Shared mapping**: Accessible by multiple processes, enabling IPC-like behavior.

Stream Abstraction Layer Stream interfaces extend basic file operations to enable buffered sequential access:

- `ifstream` (Listing C.4) reads characters sequentially, utilizing a read-ahead cache.
- `ofstream` (Listing C.5) outputs formatted strings and individual characters efficiently.

Unified I/O Model Following Unix philosophy, Apollo-RTOS unifies file and output streams. The function `do_printf` outputs formatted strings to any stream:

Listing 5.3: `include/utility/format.h`

```
template <typename ostream>
  requires requires(ostream &os, char c) { { os.putc(c) }; }
__noinline__ void do_printf(ostream &os, const char *fmt, ...);
```

File Handler Interface The `file_t` class provides a rich set of operations for file management:

Listing 5.4: `include/fs/fs_impl.h`

```
class file_t {
  fd_t *fd = nullptr;
  /* memory mapping information */

public:
  file_t(fn_t fn, size_t sz, uint32_t flags);
  ~file_t();

  void *mmap(size_t sz);

  template <typename T, typename... Args>
    requires std::constructible_from<T, Args...>
  T *mmap(Args &&...args);

  template <typename T>
  void mmap(T &obj);

  void unmap();

  template <bool sync = ASYNC>
  bool load();

  template <bool sync = ASYNC>
  size_t store() const;

  template <bool sync = ASYNC>
  size_t write(const void *buf, size_t sz, size_t off = 0);

  template <bool sync = ASYNC>
  size_t append(const void *buf, size_t sz);
```

```
31
32   template <bool sync = ASYNC>
33   size_t read(uint8_t *buf, size_t len, size_t off = 0) const;
34 };
```

5.2.3 File System Usage Example

Finally, file system operations can be utilized as follows:

Listing 5.5: `apollo/test.cpp`

```
1  void file_rw_test()
2  {
3    fn_t fn = 10;
4
5    {
6      auto file = fram::open(fn, 128, O_WRITE | O_CREATE);
7      // create private mapping
8      auto t = (char *)file.mmap(16);
9
10     const char *s = "It IS a string, ";
11     while (*s != '\0')
12       *t++ = *s++;
13     *t = '\0';
14
15     file.store();
16     // file is closed automatically by destructor
17   }
18
19   {
20     ofstream os{fn, O_CHAR_FILE};
21     // string the formatted string to the file
22     os.printf("and this is %c %s string", 'a', "formatted");
23   } // file closed automatically
24
25   {
26     ifstream it{fn};
27     while (*it != '\0') {
28       fprintf(stdout, "%c", *it); // force output to stdout
29       ++it;
30     }
31     fprintf(stdout, "\r\n");
32
33     // should output:
34     // "It IS a string, and this is a formatted string"
35   }
36 }
```

§6 Error Handling and Recovery

Linux invests significant resources into isolating, detecting, and recovering from runtime errors, whether caused by external events or internal bugs. However, on resource-constrained hardware like the Cortex-M0 or the AGC, such extensive error handling is nearly impossible due to hardware limitations. Instead, the AGC developers adopted a radically simpler approach: perform a clean system restart after any detected error.

Of course, an unexpected restart during a critical operation could leave the system in an undefined state. To mitigate this, the AGC introduced a mechanism allowing processes to register recovery routines. Similarly, Apollo-RTOS adopts this strategy: critical processes can register recovery table entries so that, if an error triggers a restart, the system can recover to a safe state.

Despite its simplicity, this strategy proves highly effective on small systems, as demonstrated by the success of the Apollo missions themselves.

6.1 Error Handling

Since the Cortex-M0 lacks a memory protection unit (MPU) for RAM, it is practically impossible to detect memory corruption caused by unauthorized writes at the hardware level—just as it was in the AGC. Consequently, error detection must be implemented entirely in software.

Apollo-RTOS follows a similar philosophy to the AGC. Error detection is performed through extensive software checks: function arguments and critical memory locations are validated using assertions. This is carried out via calls to **`assert`**, which verifies conditions and, upon failure, triggers appropriate recovery procedures.

An error can have three levels of severity:

- **`H_RESET`**: A hardware reset of the device is required.
- **`S_RESET`**: A system-wide software reset is sufficient.
- **`TERM`**: Only the faulty process must be terminated.

Listing 6.1: `include/utility/debug.h`

```
template <reset_lev_t lev, typename... Args>
void __assert(bool ex, const char *file, uint32_t line, Args &&...args)
{
  if (ex)
    return;

  if constexpr (lev == H_RESET) {
    /* if any debug statement is supplied in args, print it */
    /* copy file name and line number to registers r0, r1 */
    return trigger_hardfault();
        // print debug information and wait for manual intervention
  }

  if constexpr (lev == S_RESET) {
```

```
15      /* print debug statement */
16      return trigger_reset();
17          // cause the system to restart automatically
18    }
19
20    if constexpr (lev == TERM) {
21      return trigger_term(file, line);
22          // terminate the faulty process and carry on normally
23    }
24  }
25
26  #define assert(EX, LEV, ...) __assert<LEV>((EX), __FILENAME__, __LINE__)
```

Once a hardfault interrupt is raised, hardware pushes some registers onto the stack as described in Listing 4.6. In the **hardfault_handler** we first manually push the software saved registers Listing 4.7 onto the stack, completing the context frame. The handler then prints available debug information, offering two recovery options:

- Manual intervention by pressing <CTRL+D> to trigger a software reset.
- Performing a hardware reset by manually pressing the reset button on the board.

Listing 6.2: src/core/hardfault.cpp

```
1  __extern_C__ void hardfault_handler_body(context_t *fault_stack)
2  {
3    // disable all interrupts
4
5    if (/* hardfault was triggered by an assert statement */) {
6      /* print the source file name and line number */
7    }
8
9    /* print the context listing all register values
10    * right before the hardfault */
11
12   while (1) {
13     switch (serial::getc()) {
14     case CTRL('d'):
15       trigger_reset();
16       break;
17
18     case '?':
19       /* print a trace of the system resources */
20       break;
21     }
22   }
23 }
```

6.2 Recovery Table

When a process or a function enters a critical section, it can be unexpectedly terminated by three main causes:

- A hardware reset caused by pressing the power button,
- An unexpected loss of power, or

- A software-triggered reset following error detection.

We assume that a hardware reset is a deliberate action to restart everything from scratch. This leaves the other two scenarios, which critical sections must be protected against.

On the frontend, two RAII classes allow a process or function to set up recovery table entries:

- **`guard_proc`**: Used by a process to declare, "I am entering a critical section; if a reset occurs, restart me with this recovery data." Upon recovery, the process can decide how to proceed based on the stored data.
- **`guard_task`**: Used by a process or function to register a task that must be executed at the next startup if a reset occurs mid-critical section. This is useful for cleanup or initiating new processes.

Recovery entries can be associated with two levels:

- **`RESET`**: Recover from both a software reset and a power failure.
- **`POWER_OFF`**: Recover only from a power failure.

Listing 6.3: `include/core/recover.h`

```
struct guard_proc {
  // setup a recovery table entry at the start of the critical section
  //   to restart the current process
  guard_proc(lev_t lev, const uint8_t *data = nullptr, size_t data_sz = 0);

  // remove the entry when exiting from the critical sectino
  ~guard_proc();
};

struct guard_task {
  // setup a recovery task to be run at startup
  guard_task(lev_t lev, runnable_t task, time_t interval = 0,
             const uint8_t *data = nullptr, size_t data_sz = 0);

  ~guard_task();
};
```

Internally, the recovery mechanism is backed by two tables:

- `proc_tbl`: Contains entries for recovering processes. Each entry includes:

```
struct proc_entry_t {
  uint32_t lev; // Recovery level

  const proc_def_t *proc_def;

  uint8_t data[N_REC_DATA] = {0xFF};
  uint16_t data_sz;
};
```

- `task_tbl`: Contains entries for recovering tasks.

```
struct task_entry_t {
  uint32_t lev; // Recovery level

  runnable_t task = nullptr;
```

```
  time_t interval = 0;

  uint8_t data[N_REC_DATA] = {0xFF};
  uint16_t data_sz;
};
```

The recovery tables are stored in a file on the FRAM file system, ensuring persistence even across power failures. When an entry is updated, only that specific entry is synchronized with the file system, minimizing the cost of each transaction.

Recovery Upon detection of an error that issues a system reset via **assert**, or after a power outage, the startup code distinguishes whether it is recovering from a software reset or a power failure. It then carries out the recovery procedures according to the entries in the recovery tables.

Listing 6.4: src/core/recover.cpp

```
void recover(lev_t curr_lev) // recovery level: RESET or POWER_OFF
{
  for (auto &entry : task_tbl)
    recover(curr_lev);

  for (auto &entry : proc_tbl)
    recover(curr_lev);

  /* reset the tables */
}

struct task_entry_t {
  ...
  void recover(lev_t curr_lev)
  {
    if (!enabled(curr_lev)) // if entry is not enabled for this level
      return;

    auto ptr = copy_data();
    if (interval == 0)
      task(ptr); // immediate run the task
    else
      waitlist::reg("recover", interval, task, ptr);
        // or set up a waitlist task to be run later
  }
  ...
};

struct proc_entry_t {
  ...
  void recover(lev_t curr_lev)
  {
    if (!enabled(curr_lev))
      return;
    sched::reg_proc(proc_def, copy_data()); // schedule the process
  }
  ...
};
```

§7 Startup Sequence

When the `micro:bit` device starts, it reads the initial stack pointer from address `0x0` into the `sp` register. It then begins executing the code at the instruction address stored at `0x4`. The **`__stack`** and **`__reset`** symbols are placed at the start of the vector table along with the interrupt handlers. This table is loaded at address `0x0` by the linker script.

Listing 7.1: `src/vector_table.c`

```
__attribute((section(".vectors"))) void *__vectors[] = {
    __stack, // Initial Main Stack Pointer (MSP) value
    __reset, // Reset Handler: Entry point after power-up or reset

    ... // interrupt handlers
};
```

It is the job of the **`__reset`** function to initialize the clock, set up memory sections, initialize hardware and drivers, run tests, and finally start the scheduler.

Listing 7.2: `src/startup.cpp`

```
__extern_C__ void __reset(void)
{
  /* Activate the crystal clock */
  /* Set up memory protection for the first half of FLASH */

  SEC_INIT(data);
  SEC_ZERO(bss);          +-------------------------------+
  SEC_INIT(services);     | Initialize data sections      |
  SEC_INIT(startups);     +-------------------------------+
  SEC_INIT(apps);

  call_constructors();    // Manually call necessary constructors

  led::init();            +-------------------------------+
  serial::init();         | Set up hardware and drivers    |
  i2c::init();            +-------------------------------+
  fs::init();

  boot::init();           // Identify the type of boot

  recover::init();        +-------------------------------+
  tests::run();           | Run tests and start processes  |
  sched::init();          +-------------------------------+

  // Should never get here
  trigger_hardfault();
}
```

Initialization of Data Sections During linking, the linker places all static and global data structures into specific sections in FLASH. These must be copied to RAM at startup at predetermined addresses. Thus, the first task of **`__reset`** is to initialize these sections.

The macro **`SEC_INIT`** copies data from FLASH to RAM, while **`SEC_ZERO`** zero-initializes a section:

```
#define SEC_INIT(name) \
```

```
2   memcpy(SEC_START(name), SEC_LOAD(name), SEC_END(name) - SEC_START(name))
3
4 #define SEC_ZERO(name) \
5   memset(SEC_START(name), 0, SEC_END(name) - SEC_START(name))
```

Besides the default `.bss` and `.data` sections, we define additional custom sections:

- `.recover`: Is retained across soft resets.
- `.services`: Stores definitions for service processes (e.g., `display`, see § 8.4) started at every boot.
- `.startups`: Contains processes only launched after a hardware reset, not during recoveries.
- `.apps`: Contains processes launchable via the shell (§ 9).

The testing framework also defines `.unit_tests` and `.sys_tests` sections for test objects (§ 10).

Dynamic Global Object Construction In embedded settings, global object constructors are usually disabled for performance. However, some constructors (e.g., for **`static_list_t`**, Listing B.1) must run at startup. To support this, we manually define initializers in a special FLASH section `.init_funcs`, and call them explicitly via **`call_constructors`**.

Driver Initialization **`__reset`** then initializes modules and device drivers. These **`init`** functions set up hardware peripherals and internal data structures as needed.

Boot Type Identification As discussed in § 6.2, recovery mechanisms depend on knowing the boot type. We distinguish among:

- `FLASH` — Full flash programming occurred.
- `POWER` — Power was disconnected and reconnected.
- `BOOT` — Reset button triggered restart.
- `RESET` — Software-triggered soft reset.

Detection relies on:

- `FLASH` — Missing magic value in flash indicates reprogramming.
- `RESET` — RAM-retained magic value indicates a soft reset.
- `POWER` — Zeroed `POWER.RESETREAS` register indicates power-on reset.
- `BOOT` — Default fallback if none of the above apply.

Boot Statistics We track and store boot type statistics in a FRAM file, printed at every startup for diagnostic (and amusement) purposes:

```
boot level: flash
boot stat: n_flash = 395, n_boot = 22, n_power = 19, n_reset = 114
```

This output shows the current boot level (`flash`) and counts for different reset modes since the last FRAM filesystem format.

7.1 Scheduler Initialization

After memory, hardware devices, and drivers have been initialized, the recovery table is set up:

- If the boot type is `FLASH` or `BOOT`, the recovery table is cleanly initialized.
- Otherwise, the table is copied from its file stored in FRAM into RAM.
- If the boot type is not **`RESET`**, the `.recover` section is also copied from FLASH to RAM.

If there are tests to run, they are executed at this stage (see § 10 for details).

Finally, the scheduler is initialized:

- The idle process is registered (Listing 4.13).
- Service processes are registered.
- Startups are launched or recovery routines are executed, depending on the boot type.
- The system timer is started.
- Control transfers to the idle process, which immediately yields to the highest-priority process.

This completes the system startup.

Listing 7.3: `src/core/sched.cpp`

```
/* assign idle_task to the main process,
 * sets up service processes,
 * then enters the idle_task */
void init()
{
  intr_disable();
    // disable all interrupts, which will be enabled by the idle proc later

  // register the idle process, and mark it as the currently running process
  IDLE_PID = REG_PROC(idle, nullptr);
  cpu.curr_proc = procs[IDLE_PID];

  setup_services(); // register all service processes

  if (/* if boot type is FLASH or BOOT */) {
    // setup the startup processes from .startup section
    sched::setup_startups();
  } else {
    // run the recovery mechanisms set in the recovery table
    auto reset_lev = /* RESET or POWER_OFF */;
    recover(reset_lev); // Listing 6.4
  }

  // start the timer
  timer::init();

  // then jump to the execution of the idle task by a context switch
  // which as seen in Listing 4.13, immediately calls change_proc
  cpu.curr_proc->stk_ptr = cpu.curr_proc->stack + cpu.curr_proc->stk_sz;
```

```
30   __run(PROC_FUNC(idle), &cpu.curr_proc->stk_ptr);
31 }
```

§8 Drivers

Drivers provide abstractions for interacting with hardware components, shielding applications from low-level register operations.

8.1 Timers

The Nordic nRF51 on the `micro:bit` includes one 32-bit timer (`TIMER0`) and two 16-bit timers (`TIMER1`, `TIMER2`) [5]. Apollo-RTOS configures the 16-bit timers as 1MHz timers, generating interrupts every 1ms, inspired by Spivey [6].

- `TIMER1` — Maintains the system clock and triggers waitlist tasks (§ 4.4).
- `TIMER2` — Used experimentally for profiling.

`TIMER1`'s interrupt handler checks the waitlist periodically. Since the interrupt can occur anywhere, waitlist tasks are run on a separate stack to prevent corruption:

Listing 8.1: `src/drivers/timer.cpp`

```
__extern_C__ void timer1_handler(void)
{
  if (TIMER1.COMPARE[0]) {
    MILLIS += 1; // increment system time
    TIMER1.COMPARE[0] = 0;
  }

  if (/* task with expired deadline exsits */) {
    intr_guard guard;

    // setup temporary stk to run waitlist tasks
    auto prev_stk = (uint8_t *)get_msp();
    set_msp(stk_end);

    waitlist::run();

    // restore the original stack
    set_msp(prev_stk);
  }
}
```

Profiling Apollo-RTOS uses `TIMER2`, set up independently of the system timer, to measure CPU time spent in each process.

Every 1ms, when `TIMER2` raises an interrupt, the `timer2_handler` increments a counter associated with the currently running process. A shell command `prof` displays the runtime distribution:

```
>> prof
procs:
  |  shell: (0.864%)
  |  display: (55.567%)
  |  accel_bg: (0.34%)
  |  pkill: (0.165%)
```

```
|  prof: (1.308%)
|  htop: (14.349%)
|  idle: (27.709%)
```

One limitation is that interrupts are disabled during scheduler operations like process registration, priority changes, and context switching, making it difficult to precisely measure the CPU time consumed by processes.

8.2 Serial

The serial driver manages the UART connection and provides three main functions:

- `void busy_putc(char)`: transmits a character synchronously, busy-waiting until the transmission completes.
- `void intr_putc(char)`: transmits a character asynchronously. If a previous transmission is ongoing, the character is queued in a buffer. The `uart_handler`, triggered by a `TXDRDY` interrupt, sends queued characters. This design is inspired by Spivey [6].
- `char getc()`: waits for and receives a character synchronously.

busy_putc is used internally by **kprintf** to print debug information atomically, while user processes should use **printf**, which prints asynchronously via **intr_putc**.

Aside Apollo-RTOS allows redirecting process outputs to files via the file system's streaming mechanism. For example:

```
>> echo this is a string sent to file 10 > 10
>> cat 10
this is a string sent to file 10
```

Internally, the serial output is treated as a special file named **stdout**. Each process maintains an output file name, which defaults to **stdout**, unless specified by an output filename in shell the command: `> fn`.

This is achieved by an unified **printf** implementation, that prints to the output file of the process, be it **stdout**, or a normal file.

Listing 8.2: `include/fs/iostream.h`

```
1  template <typename... Args>
2  void fprintf(fn_t fn, Args... args);
3  // if fn == stdout, print to the serial bus using serial::intr_putc
4  // otherwise print to the file using Listing 5.3
5
6  template <typename... Args>
7  void printf(Args... args)
8  {
9    fprintf(/* output file name of curr_proc */, std::forward<Args>(args)...);
10 }
```

Receiving Inputs from UART

Apollo-RTOS uses a stack of listeners (`bool (*listener_t)(char)`) to allow processes to asynchronously receive UART characters. A process that wishes to receive input pushes a listener onto the top of this stack.

When a UART character is received, it is passed down the listener stack until a listener consumes it, at which point propagation stops. The bottom-most listener is registered by the `shell` process, so by default all incoming characters are handled by the shell.

Listing 8.3: `src/drivers/serial.cpp`

```
1  using listener_t = bool (*)(char);
2  stack<listener_t, 16> listeners; // Listing B.4
3
4  void register_listener(listener_t listener) { listeners.push(listener); }
5  void unregister_listener() { listeners.pop(); }
6
7  extern "C" void uart_handler(void)
8  {
9    if (UART.RXDRDY) {
10     char c = UART.RXD;
11
12     // send the character down the stack
13     for (auto listener: listeners)
14       if (listener(c))
15         break;
16
17     UART.RXDRDY = 0;
18   }
19
20   // ... the rest
21 }
```

8.3 I2C

The I2C master on the Nordic nRF51 enables communication with devices connected to the `micro:bit` via the I2C bus. It operates at 100KHz or 400KHz; Apollo-RTOS uses the 100KHz setting following Spivey [6].

Apollo-RTOS provides two I2C driver modes:

- **Synchronous mode:** Busy-waits for each read/write operation. Used before the scheduler starts. Also used by the recovery module to atomically write table entries to the FRAM.
- **Asynchronous mode:** Once the scheduler is active, a mutex synchronizes I2C operations across processes. While multiple interrupts occur during transmission, the process sleeps during communication, maintaining the illusion of atomicity.

The I2C communication process is as follows:

- The slave device address (`0-x030x77`) is written to `I2C0.ADDRESS`.

- Command bytes are sent sequentially, each acknowledged by an interrupt.
- After command transmission:
  - For writes: Data bytes are sent with acknowledgments.
  - For reads:
    - `STARTRX` is set.
    - If reading the last byte, `STOP` is set; otherwise, `SUSPEND` is set.
    - Data is read from `RXD` after an interrupt.
    - If more bytes are needed, `RESUME` is triggered.

The synchronous I2C communication is simple: read/write each byte following the communication protocol, and busy-wait until it finishes. The asynchronous communication is more interested, and is managed by a simple state machine with four states:

- `W_CMD`: Writing command bytes.
- `TX_DATA`: Transmitting data.
- `RX_DATA`: Receiving data.
- `NACK`: Cleanup and finalization.

Once the transfer is started by the **`xfer`** function, the calling process goes to sleep on a channel, and is woken up by the end of transfer.

Listing 8.4: `src/drivers/i2c_async.h`

```
template <bool READ>
int xfer(uint8_t addr,
         uint8_t *cmd, size_t cmd_sz,
         uint8_t *buf, size_t buf_sz);
```

`xfer` first acquires an internal mutex. It then sets up the transaction, disables `I2C_IRQ` to write initial registers, and then re-enables the interrupt before sleeping on an internal channel until the operation completes. The state machine logic is implemented in the interrupt handler (§ C.III).

8.4 Display

The 5×5 LED matrix on the `micro:bit` is arranged into three rows in a somewhat complex format. To manage it, Apollo-RTOS uses convenience macros provided by Spivey [6]. The LEDs are controlled via GPIO registers and require constant refreshing to maintain smooth animations.

The `display` driver is implemented as a low-priority service process, always started during system initialization (Listing 7.3). It periodically renders the image stored in the global variable `image`.

Processes can update the display by calling `set_image(image_t)`, which modifies the global `image` variable with the new desired display frame.

§9 Shell and Shell Commands

Apollo-RTOS includes a Unix-like shell that executes commands sent over the serial bus. The shell runs as a high-priority process that sleeps while waiting for input.

- By default, all UART inputs are forwarded to the listener registered by `shell`. This listener buffers characters until a line break (`'\r'` or `'\n'`) is received, then wakes the shell process.
- Upon waking, `shell` parses the input to extract the command name, its arguments, background execution flag, and any output redirection to a file.
- It then searches the `.apps` section for a matching process, packages the arguments into an `args_t` structure (Listing B.5), and schedules the new process with this structure as its parameter.
- Finally, it clears the input buffer and returns to sleep.

Shell commands follow the syntax:

```
cmd [arguments] [> fn] [&]
```

For example, **`echo`** `hello, world > 10` writes `hello, world` to the file `10`.

Available commands are listed using the `help` command:

```
>> help
pkill   htop    help    prof    accel   show
calc    clear   echo    cat     ls      touch
rm      heart
```

Alongside familiar Unix-like commands, Apollo-RTOS provides custom commands to demonstrate system features:

`htop` Displays running processes, waitlist tasks, and open files:

```
>> htop
boot level: flash
curr_proc: htop
  |  0. idle : (idle), [runnable]
  |  1. shell : (high), [asleep]
  |  2. display : (low), [runnable]
  |  3. accel_bg : (high), [runnable]
  |  4. htop : (medium), [running]
waitlist: NONE
fram fs:
  |  free_blks: 245, free: 31360, in use: 1408
  open files:
  |  0: size = 1024, type = char, blks: 0, 1, 2, 3, 4, 5, 6, 7
  |  5: size = 36, type = bin,  blks: 9
```

`accel` Launches a simple accelerometer-based compass on the LED matrix, pointing an arrow toward the ground. It can run in the background or foreground:

```
>> accel &
started [4] output to stdout
```

```
>> pkill accel
killed [4]

>> accel
x: 1, y: -5, z: 64
```

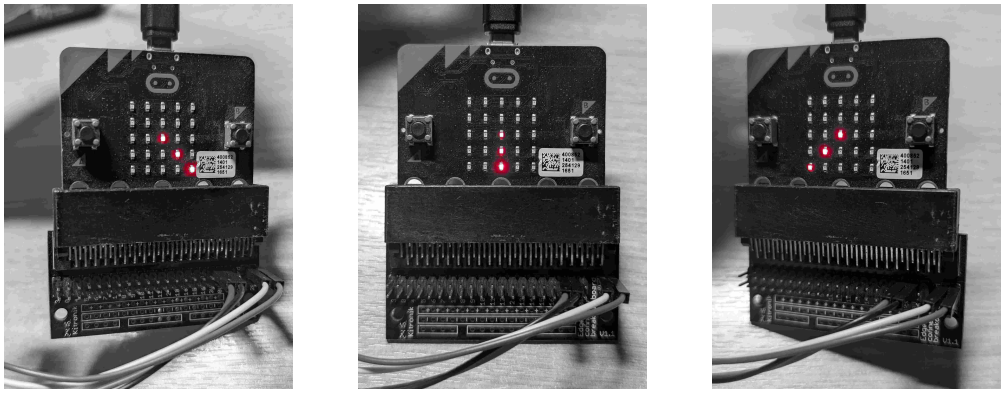

Figure 9.1: Result of `accel`

Listing 9.1: Sample workflow on shell on `minicom`

```
 _____           _ _          _____ _____ _____ _____
|  _  |___ ___| | |___   | __  |_   _|     |   __|
|     | . | . | | | . |  |    -| | | |  |  |__   |
|__|__|  _|___|_|_|___|  |__|__| |_| |_____|_____|
      |_|

Welcome to Apollo RTOS!

boot level: boot
stats: n_flash = 124, n_boot = 7, n_power = 6, n_reset = 99

>> help
pkill   htop    help    prof    accel   show
calc    clear   echo    cat     ls      touch
rm      heart

>> accel &
started [4] output to stdout

>> echo hello, world > 10

>> cat 10
hello, world

>> htop
boot level: boot
curr_proc: htop
  |  0. idle : (idle), [runnable]
  |  1. shell : (high), [asleep]
  |  2. display : (low), [runnable]
  |  3. accel_bg : (high), [runnable]
  |  4. accel : (high), [runnable]
  |  5. htop : (medium), [running]
waitlist: NONE
fram fs:
  |  free_blks: 245, free: 31360, in use: 1408
  open files:
  |  0: size = 1024, type = char, blks: 0, 1, 2, 3, 4, 5, 6, 7
  |  5: size = 36, type = bin,  blks: 9

>> rm 10

>> ls 10
file 10 not found

>> prof
procs:
  |  shell: (0.745%)
  |  display: (61.756%)
  |  accel_bg: (0.22%)
  |  prof: (0.674%)
  |  accel: (0.3%)
  |  cat: (0.11%)
  |  ls: (0.138%)
  |  rm: (0.3%)
  |  htop: (5.75%)
  |  idle: (31.567%)
```

§10 Evaluation and Testing

Apollo-RTOS includes a fully automated testing framework for unit and subsystem tests. Developing for an embedded microcontroller introduced several unique challenges in the framework's design.

- The monolithic design of the OS complicated module-level testing, as faults in one module could manifest in unrelated areas, making bug tracking difficult.
- Tests needed to run on the physical device rather than an emulator. Since copying binaries to the device is time-consuming, all tests were compiled into a single binary. The framework ensures strict isolation between tests at runtime.
- Core functionality tests (e.g., `malloc`, `file`) must run before scheduler initialization, while behavioral tests (e.g., scheduler cooperation) require an active scheduler. Full isolation between tests was necessary to prevent cascading failures across modules.

The testing framework provides a `Boost.Test`-like interface [1], using macros to register tests automatically. We first present a brief overview of the framework before discussing the tests currently available for Apollo-RTOS.

10.1 Testing Framework

Each test is described by a descriptor of the form:

```
struct test_t {
  const char *name;     // name of the test
  bool (*func)(void);   // test function to be run
  bool *passed;         // storage for the result of the test
};
```

Two dedicated sections in the binary, `.unit_tests` and `.sys_tests`, store the test descriptors. Unit tests that must run before scheduler initialization are placed in the former, and tests that require the scheduler are placed in the latter.

Tests are registered via two macros in `include/core/test.h`:

```
#define TEST(name) ...      // registers in .unit_tests
#define SYS_TEST(name) ...  // registers in .sys_tests
```

Grouped tests can be declared using:

```
#define BEGIN_SUITE(name) ...
#define END_SUITE(name) ...
```

Suites can be selectively compiled and run, e.g.,

```
cmake -DTEST_SUITES="malloc;sched_basic;ifstream" .; make tests
```

runs only the selected suites, while

```
cmake -DTEST_SUITES="" .; make tests
```

runs all tests.

At startup, before the scheduler initializes, `tests::run()` is called (Listing 7.2:22):

- `tests::run()` initializes the test sections and runs the unit tests. Each unit test:
  - Calls its test function and stores the result in `test_t::passed`, placed in the `.recovery` section (retained across soft resets).
  - Triggers a soft reset after completion to clear any residual error state.
- After unit tests, `tests::run()` schedules a process to run system tests similarly.
  - Once scheduled, the system initializes the scheduler, and system tests begin.
  - Each system test is followed by a soft reset to ensure clean isolation.

This reset-based approach ensures that each test runs in isolation, minimizing cross-test contamination. However, complete isolation cannot be guaranteed if errors in the recovery mechanism leave inconsistent states across resets.

10.2 Testing

Given the time constraints, only the most crucial aspects of the system have been tested. Below is a summary of unit and subsystem tests:

- Data structure tests: These validate core data structures by checking edge cases.
  - **`circular_buffer`**: fixed-width queue used throughout the system.
  - **`static_list_t`**: statically allocated list used by **`allocator`**.
  - **`stack`**: fixed-width stack implementation.
  - **`bitset`**: bitmap implementation used by the file system to track free blocks.
  - **`args_parser_t`**: used by `shell` to parse command-line arguments.
- Memory management tests:
  - **`allocator`**: allocation, freelist correctness, and allocation order tested.
  - **`malloc`**: process-specific memory management and cleanup validated with scheduler enabled.
  - Standard functions like **`memcpy`** and **`strcmp`** tested for correctness.
- File system tests:
  - Basic file operations (open, store) validated.
  - Mutual exclusion on file descriptors tested.
  - Streaming and output redirection to files tested.
- Scheduler tests:
  - Priority-based scheduling tested for fairness and correct behavior across sleep-/wakeup cycles.
  - Waitlist execution timing validated.

- Process concurrency through channels and mutexes tested.
- Correct execution of the testing framework itself demonstrates robustness of mutual exclusion mechanisms. In particular, tests involving multiple processes require synchronization through a minimal `barrier` built using **`mutex`**; their reliable completion without deadlocks affirms concurrency correctness.

A complete list of available tests is provided in Listing D.1.

Testing Recovery Testing the recovery mechanism could not be fully automated with the main testing framework, as its reset-before-each-test design interferes with the recovery table. Instead, dedicated binaries are compiled for recovery tests, each containing a single test case and its required processes. Currently, the following recovery test is available:

- `tests/recover/overflow`: Tests recovery after errors caused by excessive process registration. On boot, it starts 4 low-, 4 mid-, and 4 high-priority processes, with mid- and high-priority processes setting up recovery table entries. Mid-priority processes attempt to register additional low-priority processes, causing overflow. Upon restart, the test verifies that mid- and high-priority processes are correctly recovered with intact data and continue operating seamlessly.

10.3 Evaluation

Apollo-RTOS successfully demonstrates the feasibility of adapting historical embedded system concepts, such as cooperative scheduling and fault-tolerant recovery, to modern microcontrollers. The scheduler achieves predictable and deterministic process management with minimal overhead and a simple implementation, and the recovery mechanism ensures resilience against both software and hardware faults. The addition of a shell, file systems, and a testing framework provides a significant increase in usability and maintainability compared to early embedded systems.

The automated testing framework, combined with a reset-based isolation model, has proven highly effective in identifying subtle bugs during development. Core modules—including memory allocation, file system operations, process scheduling, and concurrency primitives—have been validated through extensive unit and subsystem tests. The successful recovery from simulated failures further supports the robustness of the recovery mechanism.

However, there are a number of limitations and areas for future improvement:

- Although concurrency primitives like mutexes have been tested, more extensive stress testing under high-load, real-world scenarios would provide stronger confidence in system robustness.
- Many hardware features have not been tested under adverse conditions. In particular, the I2C master device is a significant bottleneck, as all file operations on the FRAM pass through it. While our tests have covered contention scenarios on the I2C bus, testing under heavy, sustained load remains an open task.
- While shared memory IPC offers high performance, it increases the potential for race

conditions if not carefully managed. Adding more robust synchronization primitives would improve safety and ease of programming.

- The file system currently supports only fixed-size files and lacks runtime file resizing. Files are also vulnerable to corruption, as there are no built-in sanity checks on file contents; it is up to user processes to detect and handle such corruption.

  Error detection around file operations is still at an early stage. For example, corruption in the file used by the recovery mechanism is not handled robustly during startup. More work is needed to improve safety around critical file operations.

  Furthermore, startup currently lacks comprehensive file system integrity checks. If corruption is present, the system requires a manual reformatting by recompiling with specific flags. Implementing automatic file system recovery would significantly improve usability and resilience.

Overall, Apollo-RTOS successfully achieves its primary objectives of exploring the AGC's design principles in a modern context, while highlighting practical considerations for building reliable, lightweight RTOSs on resource-constrained hardware.

§11 Conclusion

Apollo-RTOS set out to explore whether the principles behind the Apollo Guidance Computer's operating system could be effectively adapted to a modern embedded platform. Through the design and implementation of a real-time operating system for the `micro:bit` v1, this project has shown that simple, well-structured scheduling and fault-tolerant techniques remain highly relevant, even on contemporary hardware.

The resulting system offers deterministic scheduling, robust error recovery, file system support, and a Unix-like shell interface, all implemented within the limited resources of the `micro:bit`. Modern C++ techniques such as RAII, templates, and type deduction helped improve both code reliability and developer productivity.

While there are still areas for improvement—such as robust file system error detection, safer primitives for IPC, and more extensive testing under load—the project successfully meets its primary objectives. It demonstrates that careful architectural choices can produce lightweight yet powerful operating systems suitable for constrained environments.

11.1 Reflection

This was my first full-fledged project in embedded systems and operating systems, where close attention to details like memory management, synchronization, and process handling was essential. A key lesson was how mistakes in foundational modules—such as memory allocation and stack setup—can lead to difficult-to-trace bugs in unrelated parts of the system.

Initially, the lack of a systematic testing framework made diagnosing such issues difficult. Unit tests for critical components like allocators and the scheduler would have caught many problems early; however, these were only added late in the project. Once the testing framework was in place, debugging and development became significantly more manageable.

Another major improvement came from adding automatic debug information dumping in the hardfault handler and error messages. Early on, the lack of detailed error reporting greatly slowed down debugging, but once proper diagnostics were implemented, it dramatically accelerated development.

Overall, this project has deepened my appreciation for rigorous engineering practices like early testing, detailed debugging, and defensive programming, all of which are critical for building reliable systems even on constrained hardware.

References

[1] Boost.test - 1.82.0, 2022. URL https://www.boost.org/doc/libs/1_82_0/libs/test/doc/html/index.html.

[2] lady ada and Michael Schroeder. Datasheet for the mb85rc256vpf fram chip, 05 2014. URL https://cdn-learn.adafruit.com/assets/assets/000/043/904/original/MB85RC256V-DS501-00017-3v0-E.pdf?1500009796.

[3] Frank O'Brien. ***The Apollo Guidance Computer: Architecture and Operation***. Springer Science & Business Media, 01 2010. ISBN 978-1-4419-0876-6. doi: 10.1007/978-1-4419-0877-3.

[4] Nordic Semiconductor. nrf51822 product specification v3.1, 2014. URL https://spivey.oriel.ox.ac.uk/bmwiki/images/7/7e/NRF51822-product-spec.pdf.

[5] Nordic Semiconductor. nrf51 series reference manual, 2014. URL https://spivey.oriel.ox.ac.uk/bmwiki/images/3/34/NRF51822-ref-manual.pdf.

[6] Mike Spivey. micro:bian. https://github.com/Spivoxity/microbian, 2021.

[7] Stephen Witt. Apollo 11: Mission out of control, 06 2019. URL https://www.wired.com/story/apollo-11-mission-out-of-control/.

§A Codebase Structure

The Apollo-RTOS codebase follows a conventional structure: header files reside in the `include/` directory, while corresponding implementations are organized under `src/`. These directories are further subdivided into logical groups based on functionality.

Application-level code, including shell commands and utilities, resides outside the core system directories. For example, the default Apollo-RTOS binary is compiled from both the `src/` and `apollo/` directories.

A brief overview of the key directories and files is presented below:

```
include
+-- core
|   +-- boot.h              # boot type definitions and access methods
|   +-- recover.h           # recovery table setup functions
|   |
|   +-- memory.h            # dynamic memory management interfaces
|   +-- sched.h             # process creation, scheduling, and modification
|   +-- waitlist.h          # waitlist task registration functions
|   |
|   +-- sync.h              # synchronization primitives (mutexes, barriers)
|   +-- signal.h            # low-overhead preemptive IPC (signals)
|   |
|   +-- hardware.h          # hardware-specific definitions
|   +-- types.h             # common types and constants
|   +-- type_traits.h       # useful C++ concepts and type utilities
|   +-- irq.h               # interrupt vector and handler definitions
|   |
|   +-- test.h              # testing framework macros
|
+-- drivers
|   +-- accel.h             # accelerometer driver interface
|   +-- display.h           # display control driver interface
|   +-- gpio.h              # GPIO control interface
|   +-- i2c.h               # synchronous and asynchronous I2C communication
|   +-- serial.h            # UART driver interface
|   +-- timer.h             # timer interrupt handling interface
|
+-- fs                      # file system core (split into modular headers)
|   +-- fs.h                # file system instantiation
|   +-- fs_bck.h            # backend: block device and inode management
|   +-- fs_impl.h           # frontend: main file system API
|   +-- fs_desc.h           # device descriptor definitions
|   |
|   +-- ifstream.h          # input file stream abstraction
|   +-- ofstream.h          # output file stream abstraction
|
+-- utility
|   +-- allocator.h
|   +-- args.h              # argument parsing utilities
|   +-- bitset.h
|   +-- circular_buffer.h
|   +-- list.h              # static linked list implementation
|   +-- stack.h
|   |
|   +-- profile.h           # experimental profiling utilities
```

```
|   |
|   +-- format.h            # formatted output declarations (do_printf)
|   +-- format_impl.h       # formatted output implementation
|   +-- debug.h             # debug/assert macros and utilities
|   |
|   +-- iostream.h          # fprintf and printf interface implementations

src
+-- core
|   +-- boot.cpp            # boot state detection implementation
|   +-- recover.cpp         # recovery mechanism and restart table management
|   |
|   +-- memory.cpp          # heap allocator, memcpy, and memory utilities
|   |
|   +-- procs.h             # process descriptor and process table definitions
|   +-- stack_allocator.h   # stack context and stack allocation utilities
|   +-- sched.cpp           # process scheduling and context switching
|   +-- waitlist.cpp        # waitlist task management
|   |
|   +-- shell.cpp           # shell interface and command listener
|   +-- test.cpp            # unit testing framework implementation
|   |
|   +-- hardfault.cpp       # hardfault handler and debug dump
|   +-- irq.cpp
|
+-- drivers
|   +-- accel.cpp           # accelerometer driver implementation
|   +-- display.cpp         # display driver implementation
|   |
|   +-- flash.cpp           # flash memory driver
|   +-- fram.cpp            # FRAM memory driver
|   |
|   +-- i2c.cpp             # I2C driver implementation (sync and async modes)
|   +-- i2c_async.h         # asynchronous I2C helper
|   +-- i2c_sync.h          # synchronous I2C helper
|   |
|   +-- serial.cpp          # UART communication driver
|   +-- timer.cpp           # timer interrupt handler implementation
|
+-- utility
|   +-- profile.cpp         # profiling tools implementation
|
+-- linker.ld.in            # linker script template
|                           # (pre-processed into linker.ld)
|
+-- mpx.s                   # context switch assembly implementation
+-- startup.cpp
+-- vector_table.c

apollo                      # Apollo-RTOS main binaries (applications)
+-- accel.cpp               # accelerometer test/utility application
+-- alph.cpp                # shell command implementations
+-- calc.cpp                # calculator application
+-- echo.cpp                # echo server/client
+-- fs.cpp                  # file system utilities
+-- heart.cpp               # heart rate monitoring app
+-- htop.cpp                # process monitoring interface

tests                       # Test binaries
```

```
|
+-- recovery                # recovery system tests
|   |
|   +-- overflow
|       +-- procs.cpp       # process table overflow test
|
+-- unit_tests              # unit tests organized by subsystem
    +-- fs
    |   +-- fs_test.cpp
    |   +-- ifstream.cpp
    |   +-- ofstream.cpp
    |
    +-- lib
    |   +-- args_parser.cpp
    |   +-- bitset.cpp
    |   +-- circular_buffer.cpp
    |   +-- list.cpp
    |   +-- stack.cpp
    |
    +-- mem
    |   +-- allocator_tests.cpp
    |   +-- malloc.cpp
    |   +-- memlib.cpp
    |
    +-- sched
    |   +-- concurrency.cpp
    |   +-- fairness.cpp
    |   +-- header.h
    |   +-- sched.cpp
    |   +-- waitlist.cpp
    |
    +-- tests.cpp
```

```
===============================================================================
 Language        Files         Lines          Code       Comments       Blanks
===============================================================================
 Assembly            1            90            50            28            12
 Autoconf            1           131           109             5            17
 C                   1           114            83            11            20
 C Header           42          4870          3774           131           965
 C++                43          4730          3718            90           922
===============================================================================
 Total              88          9935          7734           265          1936
===============================================================================
```

§B Data Structures

B.I Linked List

Listing B.1: Static linked list: `include/utility/list.h`

```
template <typename _T>
struct node_t : public _T {
  using T = _T;

  node_t *next;
  node_t *prev;

  node_t()
    requires(std::is_default_constructible_v<T>)
      : T{}, next{nullptr}, prev{nullptr}
  {
  }

  template <typename... Args>
    requires(std::is_constructible_v<T, Args...>)
  node_t(Args &&...args)
      : T{std::forward<Args>(args)...}, next{nullptr}, prev{nullptr}
  {
  }

  void detach()
  {
    if (prev)
      prev->next = next;
    if (next)
      next->prev = prev;

    next = nullptr;
    prev = nullptr;
  }
};

template <typename _node_t>
  requires requires { typename _node_t::T; } &&
           std::same_as<node_t<typename _node_t::T>, _node_t>
class static_list_t
{
private:
  using T = typename _node_t::T;

  _node_t head;
  _node_t tail;

public:
  static_list_t()
    requires(std::is_default_constructible_v<T>)
      : head{}, tail{}
  {
    head.next = &tail;
    tail.prev = &head;
  }
```

```
52
53    template <typename... Args>
54      requires(std::is_constructible_v<T, Args...>)
55    static_list_t(Args &&...args)
56        : head{std::forward<Args>(args)...}, tail{std::forward<Args>(args)...}
57    {
58      head.next = &tail;
59      tail.prev = &head;
60    }
61
62    void init()
63    {
64      new (&head) _node_t{&tail, nullptr};
65      new (&tail) _node_t{nullptr, &head};
66    }
67
68    void push_front(_node_t *node)
69    {
70      node->next = head.next;
71      node->prev = &head;
72
73      head.next->prev = node;
74      head.next = node;
75    }
76
77    void push_back(_node_t *node)
78    {
79      node->prev = tail.prev;
80      node->next = &tail;
81
82      tail.prev->next = node;
83      tail.prev = node;
84    }
85
86    _node_t *front() { return head.next; }
87    _node_t *back() { return tail.prev; }
88    bool empty() { return head.next == &tail; }
89
90  private:
91    struct iterator {
92    public:
93      using value_type = _node_t;
94      using pointer = value_type *;
95      using reference = value_type &;
96
97    public:
98      reference operator*() const { return *node; }
99      pointer operator->() const { return node; }
100
101     friend bool operator==(const iterator &lhs, const iterator &rhs)
102     {
103       return lhs.node == rhs.node;
104     }
105
106     friend bool operator!=(const iterator &lhs, const iterator &rhs)
107     {
108       return !(lhs == rhs);
109     }
110
```

```
111     iterator(_node_t *node) : node{node} {}
112
113     iterator &operator++()
114     {
115       this->node = this->node->next;
116       return *this;
117     }
118
119     iterator operator++(int)
120     {
121       auto tmp = *this;
122       ++(*this);
123       return tmp;
124     }
125
126   private:
127     pointer node;
128   };
129
130 public:
131   iterator begin() { return iterator{head.next}; }
132   iterator end() { return iterator{&tail}; }
133 };
```

B.II Allocator

Listing B.2: Memory Allocator: `include/utility/allocator.h`

```
1  struct __sz {
2    size_t sz;
3    __sz() : sz{0} {}
4  };
5
6  using chunk_t = node_t<__sz>;
7  using chunk_list_t = static_list_t<chunk_t>;
8
9  using allocator_t = byte_t *(*)(size_t);
10
11 template <allocator_t alloc_func, size_t alignment>
12 class allocator
13 {
14   chunk_list_t freelist{}; // doubly linked list
15
16 public:
17   byte_t *alloc(size_t sz)
18   {
19     if (sz == 0)
20       return nullptr;
21
22     intr_guard guard;
23
24     sz = roundup(sz, alignment);
25     auto chnk_sz = sizeof(chunk_t) + sz;
26
27     chunk_t *chunk = nullptr;
28     for (auto it = freelist.begin(); it != freelist.end(); ++it) {
29       if (it->sz >= sz && it->sz < chunk->sz)
```

```
30          chunk = &*it;
31      }
32
33      if (chunk == nullptr) {
34        chunk = (chunk_t *)alloc_func(chnk_sz);
35        chunk->sz = sz;
36      } else {
37        chunk->detach();
38      }
39
40      return (byte_t *)chunk + sizeof(chunk_t);
41    }
42
43    void dealloc(byte_t *ptr)
44    {
45      if (ptr == nullptr)
46        return;
47
48      intr_guard guard;
49      auto chunk = (chunk_t *)(ptr - sizeof(chunk_t));
50      freelist.push_back(chunk);
51    }
52  };
```

B.III Bitset

Listing B.3: Bitset: `include/utility/bitset.h`

```
1  template <size_t len>
2  class bitset
3  {
4    inline pair<uint8_t &, uint8_t> byte_and_msk(size_t idx) {
5      return {store[idx / 8], 1 << (idx % 8)};
6    }
7
8    inline pair<const uint8_t &, uint8_t> byte_and_msk(size_t idx) const {
9      return {store[idx / 8], 1 << (idx % 8)};
10   }
11
12 public:
13   bool insert(size_t idx)
14   {
15     auto [byte, msk] = byte_and_msk(idx);
16     bool exists = byte & msk;
17     if (!exists)
18       sz++;
19     byte |= msk;
20     return !exists;
21   }
22
23   bool contains(size_t idx) const
24   {
25     auto [byte, msk] = byte_and_msk(idx);
26     return byte & msk;
27   }
28
29   bool erase(size_t idx)
```

```
30   {
31     auto [byte, msk] = byte_and_msk(idx);
32     bool exists = byte & msk;
33     if (exists)
34       sz--;
35     byte &= (~msk);
36     return exists;
37   }
38
39   size_t next(size_t idx = 0) const
40   {
41     if (sz == 0)
42       return MAX<size_t>;
43
44     while (idx < len && !contains(idx))
45       idx++;
46     return idx;
47   }
48
49   size_t size() const { return sz; }
50   bool empty() const { return sz == 0; }
51
52   void set_all()
53   {
54     uint8_t msk = 0xFF;
55     for (size_t idx = 0; idx < bytes; idx++) {
56       store[idx] = msk;
57     }
58     sz = len;
59   }
60
61 private:
62   static constexpr size_t bytes = roundup(len, 8) / 8;
63
64   uint8_t store[bytes] = {(uint8_t)0};
65   size_t sz = 0;
66 };
```

B.IV Stack

Listing B.4: Stack: `include/utility/stack.h`

```
1  template <typename T, size_t N>
2  class stack
3  {
4  protected:
5    uint8_t r_arr[sizeof(T) * N];
6    size_t sz = 0;
7
8  private:
9    // iterates over the stack top to bottom
10   struct iterator {
11      /* ... */
12   };
13
14 public:
15   template <typename... Args>
```

```
  T *push(Args &&...args)
  {
    if (sz == N)
      return nullptr;

    auto t_arr = (T *)r_arr;
    auto t = new (t_arr + sz) T{std::forward<Args>(args)...};
    sz++;

    return t;
  }

  T pop()
  {
    auto t_arr = (T *)r_arr;
    return std::move(t_arr[--sz]);
  }

  T &top()
  {
    auto t_arr = (T *)r_arr;
    return t_arr[sz - 1];
  }

  bool empty() const { return sz == 0; }
  size_t size() const { return sz; }

  iterator begin() { return iterator{(T *)r_arr, sz}; }
  iterator end() { return iterator{(T *)r_arr, 0}; }
};
```

B.V Shell Command Arguments

Listing B.5: Arguments to Shell Sommands: `include/utility/args.h`

```
struct args_t {
  char str[args_len];
  mutable char *s;
  bool run_bg;

  args_t(bool run_bg) : s{str}, run_bg{run_bg} {};

  char get_option() const
  {
    /* parse the next option with the format [-c] and return the character */
  }

  template <typename T = uint32_t>
    requires std::is_unsigned_v<T>
  std::optional<T> get_uint() const
  {
    /* parse the next unsigned integer type, returning nullopt if unsuccessful */
  }

  template <typename T = int32_t>
    requires std::is_signed_v<T>
  std::optional<T> get_int() const
```

```
23   {
24     /* parse the next signed integer type, returning nullopt if unsuccessful */
25   }
26
27   const char *get_word() const
28   {
29     /* parse the next word */
30   }
31 };
```

§C Details on Implementation

C.I Malloc and Free

Listing C.1: `src/core/memory.cpp`

```
namespace kmem
{
allocator<alloc_heap, 8> kpool;

void *kmalloc(size_t sz) { return kpool.alloc(sz); }

void kfree(void *ptr)
{
  if (ptr == nullptr)
    return;
  kpool.dealloc((byte_t *)ptr);
}
} // namespace kmem

namespace heap
{
allocator<alloc_heap, 8> pool;

void *malloc(size_t sz)
{
  byte_t *ptr = pool.alloc(sz);
  /* insert chunk to the current process' allocated list */
  return ptr;
}

void free(void *ptr)
{
  if (ptr == nullptr)
    return;
  /* remove chunk from process' allocated list */
  pool.dealloc((byte_t *)ptr);
}

// called at process teardown
void cleanup(chunk_t *hd)
{
  while (hd->next != nullptr) {
    auto ptr = (byte_t *)hd->next + sizeof(chunk_t);
    /* remove hd->next from the list */
    pool.dealloc(ptr);
  }
}
} // namespace heap
```

C.II File system Data Structures

Listing C.2: File system description: `include/fs/fs_desc.h`

```
template <...>
```

```
struct fs_desc_t {
  using addr_t = /* type of address supported by device */;
    // for flash, it is uint32_t since writes must be word aligned
    // for fram, it can be uint16_t to save space

  size_t blk_sz; /* size of a block */
    // for flash, it is 1024,
    //   since that's the smallest unit we can effectively write
    // for FRAM, it is 128

  size_t max_num_blks; /* man number of blocks per file */
  size_t n_inodes; /* number of files */

  size_t n_open_files; /* number of open files */


  using fs_xfer_t = size_t (*)(addr_t addr, uint8_t *buf, size_t buf_sz);

  // methods to transfer data to and from the target device
  fs_xfer_t write, write_sync;
  fs_xfer_t read, read_sync;
};
```

Listing C.3: Filesystem backend: `include/fs/fs_bck.h`

```
template <...>
class fs_t
{
  struct hd_t {
    inode_t tbl[N_INODES];
    uint32_t magic;
    bitset<N_BLKS> free_set;
  };

  inline static hd_t hd;

public:
  // return the corresponding inode, and whether the file was just opened
  static inline pair<const inode_t *, bool> open(fn_t fn, size_t sz,
                                                 uint32_t flags)
  {
    assert(fn >= 0 && fn < desc.n_inodes, TERM);

    auto &inode = hd.tbl[fn];
    const bool in_use = inode.in_use();
    const bool to_create = flags & O_CREATE;

    if (in_use || !to_create) {
      assert(in_use, TERM, "file does not exist\r\n");
      if (sz != 0)
        debug<TRACE>("passing size (%d) to open existing file %d\r\n", sz, fn);
      return {&inode, false};
    }

    assert(to_create, TERM, "not creating non-existent file %d\r\n", fn);
    inode.set_use();

    uint8_t nblks = roundup(max(sz, BLK_SZ), BLK_SZ) / BLK_SZ;
    assert(nblks <= desc.max_num_blks, TERM);
```

```
35      const_cast<uint8_t &>(inode.nblks) = nblks;
36
37      auto success = alloc_blks(inode);
38      assert(success, TERM, "block allocation failed for file %d\r\n", fn);
39
40      if constexpr (desc.is_ram)
41        inode.set_ft(flags & O_CHAR_FILE ? CHAR : BIN);
42      else
43        inode.set_ft(BIN);
44
45      if (flags & O_PERM)
46        inode.set_perm();
47
48      inode.end = 0;
49
50      // partial update, only modifies the inode entry on the device
51      update_hd(inode);
52
53      return {&inode, true};
54    }
55
56  private:
57    // return the number of bytes written/read
58    template <auto func>
59    static inline size_t xfer(const inode_t *inode, uint8_t *buf,
60                              const size_t buf_sz, size_t offset = 0)
61    {
62      assert(inode->max_sz() >= offset + buf_sz, TERM);
63      const auto nblks = inode->nblks;
64
65      uint8_t fst_blk = offset / BLK_SZ;
66      size_t blk_offset = offset % BLK_SZ;
67      size_t fst_blk_sz = min(buf_sz, BLK_SZ - blk_offset);
68
69      uint8_t i = fst_blk;
70
71      size_t sent_sz;
72
73      auto rem = buf_sz;
74      sent_sz = func(paddr(inode->blks[i++]) + blk_offset, buf, fst_blk_sz);
75      if (sent_sz < fst_blk_sz)
76        return sent_sz;
77
78      buf += fst_blk_sz;
79      rem -= fst_blk_sz;
80
81      while (rem > 0 && i < nblks) {
82        auto blk_sz = min(rem, BLK_SZ);
83
84        sent_sz = func(paddr(inode->blks[i++]), buf, blk_sz);
85        if (sent_sz < blk_sz)
86          return buf_sz - (rem - sent_sz);
87
88        buf += blk_sz;
89        rem -= blk_sz;
90      }
91
92      return buf_sz;
93    }
```

```
94
95 public:
96   template <bool sync = ASYNC>
97   static size_t load(const inode_t *inode, void *buf, size_t buf_sz,
98                      size_t off = 0)
99   {
100    constexpr auto read = sync ? desc.read_sync : desc.read;
101    return xfer<read>(inode, (uint8_t *)buf, buf_sz, off);
102  }
103
104  template <bool sync = ASYNC>
105  static size_t store(const inode_t *inode, const void *buf, size_t buf_sz,
106                      size_t off = 0)
107  {
108    constexpr auto write = sync ? desc.write_sync : desc.write;
109    auto nbytes = xfer<write>(inode, (uint8_t *)buf, buf_sz, off);
110
111    bool prev_invalid = inode->end == 0;
112    inode->end = max(inode->end, off + nbytes);
113
114    return nbytes;
115  }
116 };
```

Listing C.4: Input file stream: `include/fs/ifstream.h`

```
1  template <auto desc>
2  class ifstream
3  {
4    using file_t = fs_impl_t<desc>::file_t;
5    file_t file;
6
7    static constexpr size_t buf_len = 16;
8    char buf[buf_len];
9    size_t buf_pos = buf_len;
10
11   size_t remaining;
12   size_t idx;
13   size_t offset;
14
15   inline void fetch()
16   {
17     size_t n_chars = min(remaining, buf_len);
18     file.read((uint8_t *)buf, n_chars, offset);
19
20     remaining -= n_chars;
21     offset += n_chars;
22     buf_pos = 0;
23   }
24
25 public:
26   ifstream(fn_t fn, size_t sz, size_t off = 0)
27       : file{fn, O_READ}, remaining{sz}, idx{sz}, offset{off}
28   {
29   }
30
31   char operator*()
32   {
33     if (idx == 0)
```

```
34        return '\0';
35
36      if (buf_pos == buf_len)
37        fetch();
38
39      return buf[buf_pos];
40    }
41
42    ifstream &operator++()
43    {
44      if (buf_pos == buf_len)
45        fetch();
46
47      idx--;
48      buf_pos++;
49      return *this;
50    }
```

Listing C.5: Output file stream: `include/fs/ofstream.h`

```
1  template <auto desc>
2  class ofstream
3  {
4    using file_t = fs_impl_t<desc>::file_t;
5    file_t file;
6
7    static constexpr size_t buf_len = 16;
8    char buf[buf_len];
9    size_t buf_pos = 0;
10
11   size_t offset;
12
13   void flush()
14   {
15     if (buf_pos != 0) {
16       file.write(buf, buf_pos, offset);
17       offset += buf_pos;
18       buf_pos = 0;
19     }
20   }
21
22 public:
23   ofstream(fn_t fn, uint32_t flags = 0)
24       : file{fn, flags | O_WRITE | O_CHAR_FILE}, offset{0}
25   {
26   }
27
28   void seek(size_t off) { offset = off == EOF ? file.fend() : off; }
29
30   void putc(char c)
31   {
32     if (buf_pos == buf_len)
33       flush();
34     buf[buf_pos++] = c;
35   }
36
37   template <typename... Args>
38   void printf(Args... args)
39   {
```

```
40      do_printf(*this, std::forward<Args>(args)...);
41    }
42  };
```

Listing C.6: File handler, file opening and closing sequence

```
1   class file_t {
2     fd_t *fd = nullptr;
3     /* memory mapping information */
4
5   public:
6
7     file_t(fn_t fn, size_t sz, uint32_t flags) {
8       fd = find_fd(fn); // first find available file descriptor
9
10      // then open the file, getting the inode
11      auto [inode, _new_file] = fs.open(fn, sz, flags);
12
13      // if the file is already open
14      if (fd->fn == fn) {
15        assert(!_new_file && fd->inode == inode, H_RESET);
16
17        fd->r_cnt++;
18        fd->w_cnt += w_en;
19        return;
20      }
21
22      // otherwise set up the file descriptor
23      fd->fn = fn;
24      fd->inode = inode;
25
26      fd->mu = mmap_unit_t{};
27      fd->r_cnt = 1;
28      fd->w_cnt = w_en;
29    }
30
31    ~file_t() {
32      if (fd != nullptr) // if it's not alrady closed
33        close();
34    }
35
36    void close() {
37      unmap();
38
39      fd->r_cnt--;
40      fd->w_cnt -= w_en;
41
42      fs.close(fd->inode);
43
44      // last reference
45      if (fd->r_cnt == 0) {
46        fd->fn = null_fn;
47        fd->inode = nullptr;
48        fd->mu = mmap_unit_t{};
49      }
50
51      fd = nullptr;
52    }
53  };
```

C.III I2C Asynchronous Communication

The communication begins in the **`xfer`** method:

Listing C.7: `src/drivers/i2c_async.h`

```
template <bool is_read>
int xfer(uint8_t addr,
         uint8_t *cmd, size_t cmd_sz,
         uint8_t *buf, size_t buf_sz)
{
  // acquire lock before continuing
  mtx.lock();

  // ... copy the buffer addresses and buffer sizes

  // initial state is W_CMD
  state.stage = stage_t::W_CMD;
  I2C0.ADDRESS = addr;

  // start transmission and send the first byte
  I2C0.STARTTX = 1;
  I2C0.TXD = *cmd;

  // put the current process to sleep until woken up after completion
  sched::wait(intr_chan);

  // ... report any error that might've occured
}
```

This method just starts the transmission, and puts the process to sleep. The state machine is implemented by the interrupt handler:

Listing C.8: `src/drivers/i2c_async.h`

```
void handler(void)
{
  intr_guard guard{I2C0_IRQ};

  if (I2C0.ERROR)
    // ... if there was an error, cancel the communication

  else if (state.stage == stage_t::W_CMD) {
    // ... receive acklowledgement
                                                    +------------+
    // still writing the command                    |  W_CMD     |
    if (state.cmd_idx < state.cmd_sz)                +------------+
      // ... send next byte

    // no data to send or receive, so stop
    else if (state.data_sz == 0)
      // ... stop

    // if we're writing
    else if (state.mode == mode_t::WRITE) {
      state.stage = stage_t::TX_DATA;
      // ... send the first data byte
    }

```

```
    // we're reading
    else {
      state.stage = stage_t::RX_DATA;
      // ... set the SHORTS register and STARTRX = 1
    }
  }

  else if (state.stage == stage_t::TX_DATA) {
    // ... receive acklowledgement
                                                          +------------+
    // finished writing                                   |  TX_DATA   |
    if (state.data_idx == state.data_sz)                  +------------+
      // ... stop

    // more bytes to write
    else {
      state.stage = stage_t::TX_DATA;
      // ... send the next byte
    }
  }

  else if (state.stage == stage_t::RX_DATA) {
    // ... receive acklowledgement
                                                          +------------+
    // ... copy byte from I2C0.RXD                        |  RX_DATA   |
                                                          +------------+
    // finished reading, move to NACK
    if (state.data_idx == state.data_sz)
      state.stage = stage_t::NACK;

    // continue reading
    else {
      state.stage = stage_t::RX_DATA;
      // ... set the SHORTS register and RESUME = 1
    }
  }

  else if (state.stage == stage_t::NACK) {
    // ... receive acklowledgement                      +------------+
                                                        |  NACK      |
    mtx.unlock(); // release the lock                   +------------+
    sched::notify(intr_chan); // then wake up the waiting process
  }
}
```

With this implementation, multiple processes can communicate over the I2C bus without conflicts. All requests are serialized using the **`mutex`**, ensuring orderly access to the bus. Additionally, while one process is communicating over the I2C bus, it does not monopolize the CPU. Instead, it allows other processes to continue executing, improving overall system efficiency.

§D Tests

Listing D.1: Available tests to Apollo RTOS, all passing

```
args_parser:
    all_cases                       # tests for argument parsing correctness

bitset:
    basic_operations                # bitset creation, and basic bitwise operations
    iteration                       # iteration over set bits

circular_buffer:
    basic_operations                # push/pop operations in normal usage
    capacity_and_overwrite          # behavior when buffer capacity is exceeded
    edge_cases                      # handling full/empty buffers and boundary
    conditions
    wrap_around_and_iteration       # correct behavior during wrap-around scenarios

stack:
    basic_integer_operations        # push/pop with integer data
    edge_cases                      # handling empty stack, overflow prevention
    order_preservation              # LIFO (Last-In-First-Out) behavior verification
    struct_operations               # storing and manipulating struct data
    template_specialization         # tests for templated stack specializations

static_list:
    basic_operations                # insert, remove, and traversal operations
    edge_cases                      # boundary tests such as empty list handling
    iteration_functionality         # iterator support and traversal correctness
    node_management                 # memory reuse and node handling tests

allocator:
    basic_allocation_and_reuse      # basic alloc/free scenarios and reuse
    edge_cases                      # allocating 0 bytes, handling invalid frees
    freelist_management             # freelist integrity after allocs and deallocs
    memory_reuse_scenarios          # stress tests for memory reuse patterns
    rounding                        # block size rounding correctness tests

malloc:
    basic_malloc_free               # basic malloc/free cycle validation
    cleanup_after_process           # automatic memory cleanup on process exit
    complex_pattern_with_cleanup    # malloc/free with cleanup validation
    null_and_zero_handling          # behavior when allocating 0 bytes or NULL
    freeing
    process_isolation               # memory isolation between concurrent processes

memlib:
    memcmp_test                     # memory comparison correctness
    memcpy_test                     # memory copy correctness
    memset_test                     # memory set (initialization) correctness
    strcmp_test                     # string comparison correctness
    strcpy_test                     # string copy correctness

file_system:
    fs_basic:
        mmap                        # memory-mapped file I/O tests
        multi_blk_file              # multi-block file allocation and access
        open_store_close            # file open, store, and close cycle verification
```

```
    fs_contention:
        write_contention            # concurrent file writing contention scenarios
    fs_mapping:
        unshared_mapping            # behavior of private (unshared) file mappings
    ifstream:
        basic_read_write            # sequential read/write correctness
    ofstream:
        basic_write                 # basic output file stream functionality
        write_redirection           # output redirection and formatted writes

scheduler:
    sched_basic:
        sleep                       # sleep and wakeup scheduling tests
        channel_wakeup              # wakeup through communication channels
        priority_order              # priority-based task execution ordering
        round_robin                 # fairness among equal-priority tasks
        timer_duration              # sleep duration validation
    sched_concurrency:
        sleep_notify                # IPC through channel
        mutex_exclusion             # mutual exclusion via mutexes
    sched_fairness:
        sleep_fairness              # fairness among sleeping tasks
        yield_fairness              # fairness during voluntary yielding
    waitlist:
        waitlist_basic              # basic task scheduling using the waitlist
        waitlist_fs_op              # waitlist usage during file system operations
```